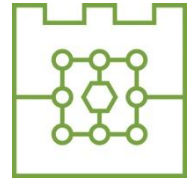




**Politechnika Krakowska
im. Tadeusza Kościuszki**
Wydział Informatyki i Telekomunikacji



Michał Bąk
Numer albumu: 144753

**Przewidywanie zwłóknienia wątroby z wykorzystaniem
metod uczenia maszynowego**

Prediction of liver fibrosis using machine learning methods

**Praca inżynierska
na kierunku Informatyka**

Praca wykonana pod kierunkiem:
dr inż. Paweł Jarosz

Kraków, 2025

Spis treści

1. Wstęp.....	- 3 -
1.1 <i>Rozwój technologii w medycynie</i>	<i>- 3 -</i>
1.2 <i>Wykorzystanie danych medycznych w predykcji chorób</i>	<i>- 4 -</i>
1.3 <i>Cel i zakres pracy</i>	<i>- 4 -</i>
2. Przegląd literatury.....	- 6 -
3. Schemat eksperymentu	- 7 -
4. Analiza zbioru danych	- 9 -
4.1 <i>Charakterystyka danych.....</i>	<i>- 11 -</i>
4.2 <i>Struktura i format danych</i>	<i>- 11 -</i>
4.3 <i>Wstępne przetwarzanie danych.....</i>	<i>- 12 -</i>
4.4 <i>Analiza eksploracyjna danych</i>	<i>- 13 -</i>
5. Budowa modeli klasyfikacyjnych.....	- 26 -
5.1 <i>Wybrane algorytmy</i>	<i>- 26 -</i>
5.2 <i>Optymalizacja modeli</i>	<i>- 27 -</i>
5.3 <i>Implementacja</i>	<i>- 28 -</i>
6. Ewaluacja modeli	- 33 -
7. Wyniki i dyskusja	- 35 -
7.1 <i>Wyniki.....</i>	<i>- 36 -</i>
8. Podsumowanie.....	- 45 -
8.1 <i>Przegląd zastosowanych algorytmów.....</i>	<i>- 45 -</i>
8.2 <i>Wnioski dotyczące metod optymalizacji.....</i>	<i>- 46 -</i>
8.3 <i>Znaczenie zwiększenia zbioru danych.....</i>	<i>- 46 -</i>
8.4 <i>Kierunki dalszego rozwoju i rekomendacje</i>	<i>- 46 -</i>
8.5 <i>Podsumowanie końcowe.....</i>	<i>- 47 -</i>
Bibliografia.....	- 49 -
Spis rysunków	- 51 -
Spis tabel.....	- 52 -

1. Wstęp

Coraz częściej słyszę o próbach wykorzystania modeli sztucznej inteligencji w rozwiązaniach medycznych. Sebastian Patrzyk utrzymuje, że sztuczna inteligencja (ang. *artificial intelligence*) w zastraszającym tempie rewolucjonizuje wiele dziedzin, również medycynę. Autor przyznaje, że analiza dużych zbiorów danych, analiza genomów czy przyspieszanie rozwoju leków poprzez odkrywanie nowych molekuł (np. Chemistry42) to zdecydowane zalety stosowania sztucznej inteligencji [1].

W mojej pracy pochyliłem się nad tematem wykorzystania metod uczenia maszynowego w diagnostyce chorób, mówiąc ściślej na rozpoznawaniu i ocenie stopnia zwłóknienia wątroby. Wybrałem to zagadnienie, ponieważ aktualnie w diagnostyce zwłóknienia wątroby stosuje się proces zwany biopsją. Pomimo tego, że biopsja wątroby pozwala na skuteczną ocenę stopnia zwłóknienia wątroby to jest zabiegiem inwazyjnym oraz może prowadzić do dalszych powikłań [2]. Dlatego uważam, że poszukiwanie modelu sztucznej inteligencji, który będzie precyzyjnie określał stopień zwłóknienia wątroby w dodatku będzie szybszy od tradycyjnej biopsji oraz nie powodował potencjalnych powikłań, jest tematem, nad którym warto się skupić.

Problem stworzenia modelu sztucznej inteligencji w celu diagnostyki chorób wątroby jest znany i można znaleźć prace, które poruszają ten temat. W swoim artykule Marek Matacz opisuje jak sztuczna inteligencja dość precyzyjnie wykrywała niealkoholową tłuszczeniową chorobę wątroby [3]. W innych badaniach z wykorzystaniem sztucznej inteligencji, autorzy skupiają się na analizie obrazów medycznych (np. tomografia) [4]. Mimo to nie natknąłem się na rzetelne porównanie różnych metod uczenia maszynowego wytrenowanych na jednym, wspólnym zbiorze danych, który jest odpowiednio przygotowany, aby zostać wykorzystany jako zbiór, na którym będą trenowane modele sztucznej inteligencji.

W dalszej części pracy wykorzystuje dziesięć różnych algorytmów uczenia maszynowego, dwie techniki optymalizacji parametrów. Modele trenuje na zbiorze HCV-Egy-Data, który przetwarzam w celu łatwiejszego uczenia modeli, co się przekłada na uzyskanie lepszych rezultatów [5]. Dodatkowo, aby uzyskać jeszcze lepsze wyniki problem 4 klasowej klasyfikacji sprowadzam do klasyfikacji binarnej (zdrowy lub chory).

W poszczególnych rozdziałach pochyliam się nad tematami takimi jak wstępny proces przetwarzania danych, trenowanie modeli, optymalizacja modeli za pomocą hiperparametrów, metryki oceny skuteczności modeli oraz dalsza dyskusja nad uzyskanymi wynikami i perspektywami rozwoju.

Przed wszystkim skupiłem się na wytrenowaniu i porównaniu trzydziestu modeli oraz wybraniem najlepszych spośród nich, które mogłyby wspomóc lekarzy w diagnozowaniu stopnia zwłóknienia wątroby. Na końcu zwrócę uwagę na modyfikacje, które mogą prowadzić do jeszcze lepszych rezultatów.

1.1 Rozwój technologii w medycynie

W ostatnich latach dostrzegam wpływ sztucznej inteligencji zarówno w sektorze medycyny oraz w życiu codziennym za sprawą chat-GPT. W 2021 był taki przypadek, gdzie matka czteroletniego Alexa nie mogła uzyskać pomocy od kilkunastu lekarzy przez ponad 3 lata. W końcu zwróciła się z pytaniem do chata-GPT. Chatbot po przeanalizowaniu objawów opisanych przez matkę oraz dołączonych wyników badań zidentyfikował rzadką chorobę neurologiczną u dziecka, którą później potwierdził lekarz [6]. Przykład ten pokazuje, że sztuczna inteligencja może wspierać lekarzy w diagnozowaniu chorób.

Początki wykorzystywania sztucznej inteligencji w medycynie sięgają lat 70 ubiegłego wieku, kiedy to został opracowany system MYCIN do diagnozowania zakażeń krwi [7]. Od tego czasu minęło ponad 50 lat. W tym czasie za sprawą wykorzystania modeli sztucznej inteligencji poprawie uległy wyniki badań oraz koszty prowadzenia terapii. Szczególnie chciałbym zwrócić tutaj uwagę na sektor telemedycyny, który znacząco zyskał za sprawą nowoczesnych technologii informatycznych. Mam na myśli zdalne konsultacje, które zyskały na popularności szczególnie podczas pandemii COVID-19 oraz monitoring pacjentów, który to pozwala lekarzom na ciągłą obserwację stanu zdrowia pacjenta [8]. Lata 80. i 90. Uznaje się za początek digitalizacji dokumentacji medycznej.

Trudno jest być obojętnym na nie tylko teoretyczne, ale faktyczne zastosowanie sztucznej inteligencji w medycynie. Sztuczna inteligencja może pochwalić się takimi sukcesami jak wczesne wykrywanie nowotworów, terapie dostosowane do indywidualnych potrzeb pacjentów czy znacznie przyspieszony proces odkrywania i wprowadzania nowych leków, w tym i szczepionek [9, 10].

Gwałtowny rozwój sztucznej inteligencji i jej zastosowanie w medycynie niesie sporo korzyści. Jednakże prawo polskie nie nadąża za tak dynamicznym rozwojem. W wywiadzie Anna Gawrońska twierdzi, że Unia Europejska uznaje systemy sztucznej inteligencji za „wysokiego ryzyka”, tym samym nakazując przeprowadzenie testów oraz ich dokładnej certyfikacji. Anna Gawrońska wskazuje parę zagrożeń wynikających z tak szybkiego tempa rozwoju sztucznej inteligencji, mianowicie martwi się redukcją kadr w wyniku automatyzacji, problemami z odpowiedzialnością za błędy oraz ryzykiem naruszenia prywatności i bezpieczeństwa danych pacjentów [11]. Mimo to, trudno mi sobie wyobrazić współczesną medycynę bez zastosowania sztucznej inteligencji, która może wspierać zarówno lekarzy jak i pacjentów.

1.2 Wykorzystanie danych medycznych w predykcji chorób

Współczesna medycyna gromadzi różne dane, takie jak dane opisowe (subiektywne odczucia), dane liczbowe (badania laboratoryjne), sygnały (puls), obrazy i zdjęcia [12]. Jest również możliwość gromadzenia danych za pomocą urządzeń typu wearables, czyli opasek fitness i zegarków sportowych, które zbierają takie dane jak tętno czy poziom tlenu we krwi. Dzięki tym zabiegom powstają obszerne zbiory danych, które następnie mogą zostać udostępnione i wykorzystane jako baza wiedzy do trenowania modeli sztucznej inteligencji. Gromadzenie oraz udostępnianie takich danych medycznych uważam za pozytywny aspekt, ponieważ jeżeli w takim zbiorze danych znacząco wzrośnie liczba pacjentów, którzy utrzymują, że mają „nietypowy kaszel”, może świadczyć to o zagrożeniu epidemią [13].

W mojej pracy przy tworzeniu modeli predykcyjnych istotne są dla mnie wyniki badań laboratoryjnych, ponieważ na ich podstawie modele szukają wzorców, które wskazują na konkretne stadia choroby. Skupiając się na włóknieniu wątroby dużą wagę przykładam do parametrów biochemicznych (np. poziom enzymu ALT), ponieważ pozwalają one na przewidywanie marskości wątroby. Zdecydowałem się skorzystać ze zbioru HCV-Egy-Data, ponieważ zawiera on zarówno parametry kliniczne jak i subiektywne odczucia pacjentów.

1.3 Cel i zakres pracy

Moja praca polega na stworzeniu oraz porównaniu trzydziestu modeli sztucznej inteligencji oraz wyselekcjonowanie spośród nich najlepszych algorytmów uczenia maszynowego, które mogłyby wesprzeć prace lekarzy. Aby osiągnąć ten cel wykorzystam zbiór HCV-Egy-Data, który zawiera wiele rekordów z pacjentami oraz sporo informacji o każdym z

nich - od wyników badań biomechanicznych, poprzez subiektywne odczucia, aż po dane demograficzne. W związku z tym, że biopsja wątroby jest inwazyjną i czasochłonną metodą diagnostyczną, wychodząc na przekór proponuje podejście nieinwazyjne oraz mniej czasochłonne dbając przy tym o komfort badanego.

Zakres pracy obejmuje następujące etapy:

1. Analiza wstępna danych:
 - Przygotowanie zbioru danych do analizy poprzez standaryzację, usunięcie wartości odstających oraz transformację etykiet na format umożliwiający ich przetwarzanie.
 - Wizualizacja danych w celu identyfikacji wzorców oraz zależności pomiędzy zmiennymi.
2. Wybór algorytmów i budowa modeli predykcyjnych:
 - Zastosowanie różnych algorytmów uczenia maszynowego do predykcji stadiów chorób wątroby.
 - Optymalizacja hiperparametrów modeli za pomocą metod RandomizedSearchCV oraz Optuna.
3. Ocena skuteczności modeli:
 - Wykorzystanie metryk takich jak dokładność, precyzja, czułość, obszar pod krzywą ROC (AUC-ROC) oraz wykres SHAP do porównania wydajności zastosowanych algorytmów.
 - Interpretacja wyników na podstawie macierzy pomyłek oraz krzywych ROC.
4. Wizualizacja wyników:
 - Opracowanie wykresów prezentujących wyniki analizy danych oraz wydajność algorytmów.
 - Przedstawienie wpływu poszczególnych cech na predykcję chorób.
5. Formułowanie wniosków:
 - Ocena możliwości zastosowania algorytmów uczenia maszynowego w rzeczywistych scenariuszach medycznych.
 - Wskazanie potencjalnych obszarów dalszych badań i rozwoju analizowanych metod.

Chciałbym, aby ta praca odpowiedziała na pytanie, które modele sztucznej inteligencji najlepiej radzą sobie w diagnozowaniu stadiów choroby wątroby w oparciu o posiadane dane. Zależy mi również na procesie optymalizacji modeli poprzez hiperparametry. Opowiem również o technicznych aspektach analizy danych i przetwarzania danych.

2. Przegląd literatury

Zastosowanie uczenia maszynowego w medycynie stwarza ogromne możliwości dla tego sektora. Rośnie liczba publikacji poruszających temat zastosowania algorytmów sztucznej inteligencji w celu poprawy jakości i szybkości diagnozowania oraz prowadzenia terapii. Korzystam również z faktu rosnącej popularności wykorzystania uczenia maszynowego, wykorzystując publicznie dostępne zbiory danych do tworzenia modeli.

Najpierw posłużę się przykładem Platformy Analiz Medycznych (CMAP) firmy Comarch, która wspiera proces diagnostyczny oraz usprawnia zarządzanie placówkami medycznymi. Platforma pozwala m.in. na analizę Elektronicznej Dokumentacji Medycznej, monitorowanie stanu pacjenta, a także wczesne wykrywanie zaburzeń takich jak migotanie przedsionków czy częstoskurcze komorowe [14].

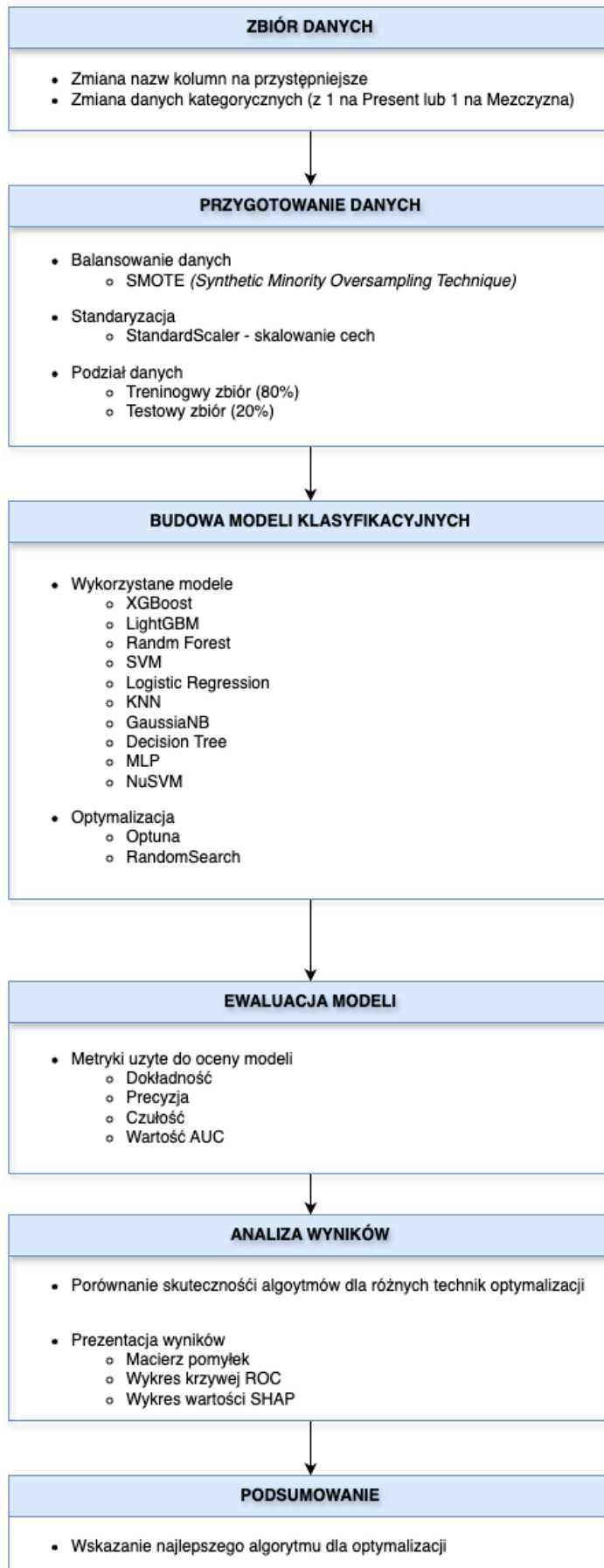
Warto też zwrócić uwagę na wykorzystanie sztucznej inteligencji w kontekście przewidywania zgonów na podstawie EKG, gdzie lekarze wykazali się mniejszą skutecznością w ocenie ryzyka zgonu niż modele sztucznej inteligencji. Modele sztucznej inteligencji znalazły zastosowanie w identyfikacji molekuł o silnych właściwościach antybakteryjnych przyczyniając się do wynalezienia nowych antybiotyków [15]. Te zastosowania pokazują, że sztuczna inteligencja może się znacząco przyczynić do zrewolucjonizowania sektora medycznego.

Krokiem milowym było wprowadzenie metod uczenia głębokiego (ang. *deep learning*), czyli technologii, która imituje działanie ludzkiego mózgu, wykorzystując sieci neuronowe. Deep learning wyróżnia się na tle machine learning m.in. tym, że pracuje na jeszcze większych zbiorach danych i jest bardziej samodzielny, tzn. wymaga rzadszej korekty ze strony człowieka. Mimo że sam proces uczenia jest dosyć podobny w obu przypadkach, to zalety przemawiają na korzyść deep learningu. Deep learning oferuje efektywniejsze rozwiązania – na przykład w kryminalistyce umożliwia precyzyjniejsze rozpoznawanie osób poszukiwanych, ponieważ proces wymaga prześledzenia setek tysięcy zdjęć i nagrań z kamer. Bez zaawansowanych metod uczenia maszynowego byłoby to czasochłonne, a organy ścigania, podobnie jak sektor medyczny, działają pod presją czasu [16].

Podsumowując, w dzisiejszych czasach zarówno deep learning, jak i machine learning przynoszą korzyści w wielu dziedzinach, nie tylko medycznych, ale również np. w kryminologii. Uważam, że dalsze pogłębianie wiedzy w tym zakresie przyniesie jeszcze większe udogodnienia w codziennym życiu i pracy. Pozostając w temacie medycyny, twierdzę, że sztuczna inteligencja poprawi jakość opieki nad pacjentami i usprawni pracę lekarzy w bliskiej przyszłości.

3. Schemat eksperymentu

Rysunek [1] przedstawia schemat eksperymentu. Rysunek w sposób obrazowy przedstawia kolejne etapy, które podjąłem w celu osiągnięcia ustalonego celu pracy. Z rysunku można odczytać, że prace zacząłem od zmiany nazw kolumn na bardziej przystępne (usuwając puste znaki, gdzie było to konieczne) oraz zmiany cech kategorycznych. Następnie swoją uwagę poświęciłem na przygotowanie danych, czyli zbalansowanie danych za pomocą SMOTE, standaryzacji cech oraz podziałem danych na zbiory treningowe i testowe. W dalszej części zająłem się projektowaniem modeli oraz ich optymalizacją w wyniku czego powstało trzydzieści modeli. Następnie wyznaczyłem metryki względem których będę mierzył efektywność modeli, by na końcu dokonać analizy uzyskanych wyników i zastanowić się nad ewentualnymi usprawnieniami.



Rysunek 1 - schemat eksperymentu

4. Analiza zbioru danych

Poszukując zbioru danych do mojej pracy, stanąłem przed pytaniem, który zbiór danych byłby odpowiedni. Zdecydowałem się na zbiór danych HCV-Egy-Data, ponieważ umożliwił mi analizę oraz klasyfikację różnych stadiów choroby wątroby. W zbiorze tym są zgromadzone dane pochodzące od egipskich pacjentów, którzy przechodzili leczenie HCV (wirus zapalenia wątroby typu C) przez około 18 miesięcy. Co ciekawe Egipt ma jeden z najwyższych wskaźników zakażeń wirusem HCV, szczególnie w delcie Nilu, co jeszcze bardziej skłoniło mnie do wyboru tego zbioru [17]. Dostateczna liczba pacjentów oraz atrybutów, różnorodność cech medycznych oraz publiczna dostępność zbioru, wpłynęły pozytywnie na moją decyzję o wyborze właśnie tego konkretnego zbioru danych. W moim odczuciu ten zbiór pozwala mi na przeprowadzenie rzetelnej analizy oraz dalszych etapów eksperymentu.

Tabela [1] przedstawia cechy ze zbioru danych wykorzystane w pracy a także statystyki opisowe takie jak, średnia, mediana, moda, odchylenie standardowe, wartość minimalna oraz wartość maksymalna.

Nazwa cechy	Polskie tłumaczenie	Średnia	Mediana	Moda	Odchylenie standardowe	Min	Max
Age	Wiek	46	46	56	0.5	32	61
Gender	Płeć	1.5	1	1	4.1	1	2
BMI	BMI	28	29	34	0.5	22	35
Fever	Gorączka	1.5	2	2	0.5	1	2
Nausea/Vomiting	Nudności/Wymioty	1.5	2	2	0.5	1	2
Headache	Ból głowy	1.5	1	1	0.5	1	2
Diarrhea	Biegunka	1.5	2	2	0.5	1	2
Fatigue	Zmęczenie	1.5	1	1	0.5	1	2
Jaundice	Żółtaczka	1.5	2	2	0.5	1	2
Epigastric pain	Ból w nadbrzuszu	1.5	2	2	0.5	1	2
WBC	Liczba białych krwinek	7533	7498	3271	2668	2991	12101
RBC	Liczba czerwonych krwinek	4422130	4438465	4153369	346358	3816422	5018451
HGB	Poziom hemoglobiny we krwi	12	13	15	2	10	15
Plat	Liczba płytek krwi	158348	157916	103368	38795	93013	226464
AST_1	Poziom enzymu AST w pierwszym tygodniu	83	83	86	26	39	128
ALT_1	Poziom enzymu ALT	84	83	126	26	39	128

	w pierwszym tygodniu						
ALT_4	Poziom enzymu ALT w czwartym tygodniu	83	82	120	26	39	128
ALT_12	Poziom enzymu ALT w dwunastym tygodniu	84	84	39	26	39	128
ALT_24	Poziom enzymu ALT w dwudziestym czwartym tygodniu	84	83	115	26	39	128
ALT_36	Poziom enzymu ALT w trzydziestym szóstym	83	84	64	26	5	128
ALT_48	Poziom enzymu ALT w czterdziestym ósmym tygodniu	83	83	78	26	5	128
ALT after 24w	Poziom enzymu ALT po 24 tygodniach od zakończenia leczenia	33	34	34	7	5	45
RNA Base	Poziom RNA przed leczeniem	590951	593103	23179	353935	11	1201086
RNA 4	Poziom RNA w czwartym tygodniu	600896	597868	709348	362315	5	1201715
RNA 12	Poziom RNA w dwunastym tygodniu	288754	234359	5	285351	5	3731527
RNA EOT	Poziom RNA na końcu leczenia	287660	251376	5	264560	5	808450
RNA EF	Poziom RNA na końcu terapii	891378	244049	5	267701	5	810333

Baseline histological grading	Stopień nasilenia stanu zapalnego wątroby	10	10	15	4	3	16
Baseline histological staging	Ocena stopnia zwłóknienia wątroby	2.5	3	4	1	1	4

Tabela 1 - Charakterystyka zbioru danych

4.1 Charakterystyka danych

Zbiór HCV-Egy-Data dostarcza szczegółowe oraz zróżnicowane dane medyczne. Zbiór zawiera ponad 1300 rekordów oraz 29 kolumn, każdy rekord to jeden pacjent, a każda z kolumn dostarcza informacje na temat stanu zdrowia pacjenta. Zbiór jest zbalansowany, tzn. klasa kategoryzująca (`baselinehistological_staging`) zawiera w przybliżeniu równą ilość dla każdego ze stadiów tj. 336 próbek dla I stadium, 332 próbki dla II stadium, 355 próbek dla III stadium oraz 362 próbki dla IV stadium. Dzięki temu, że zbiór jest zbalansowany mogę przeprowadzić analizę statystyczną oraz zaprojektować różne modele predykcyjne.

Zbiór HCV-Egy-Data jest zróżnicowany pod względem zebranych informacji, znajdują się w nim m.in.:

- Atrybuty demograficzne, takie jak wiek, płeć, wskaźnik BMI,
- Informacje o objawach klinicznych: gorączka, nudności/wymioty, bóle głowy, zmęczenie,
- Parametry biochemiczne takie jak liczba leukocytów (WBC), liczba erytrocytów (RBC), poziom hemoglobiny (HGB),
- Kategoria docelowa (`baselinehistological_staging`), czyli stan wątroby podzielony na cztery stadia.

Jako że dane mają medyczny charakter to zwracam uwagę na:

- Jakość i spójność – sprawdzam, czy występują duplikaty oraz obserwacje odstające (ang. Outliners), mogące zniekształcać wyniki,
- Brakujące wartości – choć deklarowane jest, że ich nie ma, weryfikuję to w trakcie wstępnego przetwarzania,
- Rozkład klas – oryginalnie zmienna docelowa przyjmuje kilka kategorii, co może wymagać zastosowania metod równoważenia zbioru, np. SMOTE.

Wszystkie cechy w zbiorze prezentuje tabela [1], wraz z polskim tłumaczeniem w celu lepszego zrozumienia zgromadzonych informacji.

4.2 Struktura i format danych

Korzystam ze zbioru HCV-Egy-Data w formacie CSV z uwagi na prostotę i szybkość wczytywania danych do narzędzi (np. biblioteka `pandas` w pythonie). Jest to również format zalecany dla zbiorów udostępnionych publicznie [18]. Odnosnie struktury zbiór zawiera zmienne kategoryczne, takiej jak płeć czy obecność danych objawów oraz zmienne liczbowe, np. enzymy wątrobowe i wskaźniki krwi. Takie zróżnicowanie danych wymaga ode mnie

skupienie się na metodach wstępnego przetwarzania danych, w tym chociażby standaryzacji, usunięcia duplikatów czy usunięcia wartości odstających.

Do najważniejszych cech zbioru zaliczam:

- Rozmiar i liczba atrybutów – zbiór liczy ponad 1300 rekordów i prawie 30 kolumn,
- Spójność – brak pustych wartości w kolumnach, co minimalizuje ryzyko błędów,
- Zmienna docelowa – oryginalnie klasa jest wieloklasowa, aczkolwiek sprowadzam problem do zadania binarnej klasyfikacji (zdrowy/chory).

Uważam, że takie podejście pozwala na łatwiejszą implementację różnorodnych algorytmów uczenia maszynowego oraz późniejsze porównanie ich wyników. Z uwagi na to, że każdy algorytm ma takie same warunki uzyskane wyniki powinny być rzetelne.

4.3 Wstępne przetwarzanie danych

Wstępne przetwarzanie danych to dla mnie kluczowy etap, ponieważ skrupulatne podejście w tej fazie ma znaczący wpływ na wyniki uzyskane przez modele. Więc w kontekście wstępnego przetwarzania danych skupiłem się na:

1. Zmianie nazw kolumn
2. Redukcji wieloklasowej zmiennej docelowej do zadania binarnej klasyfikacji
3. Eliminacji duplikatów
4. Identyfikacji i usunięciu obserwacji odstających
5. SMOTE (równoważenie zbioru)
6. Standaryzacji

☐ Zmiana nazw kolumn

Niektóre nazwy kolumn mają zbyt długie nazwy, zawierają puste znaki na końcu nazw cech lub niejednoznaczne. Ujednoliciłem oraz skróciłem nazwy atrybutów, w celu łatwiejszego odwoływania się do nich. Na przykład ujednoliciłem nazwę atrybutu ‘AST 1’ informujący o poziomie enzymu AST u pacjenta w pierwszym tygodniu od rozpoczęcia leczenia na ‘AST_1’, a nazwę atrybutu ‘ALT4’ zmieniłem na ‘ALT_4’.

☐ Redukcja wieloklasowej zmiennej docelowej do zadania binarnej klasyfikacji

Cecha ‘Baselinehistological staging’ przyjmuje cztery kategorie, które są stadiami choroby wątroby. Scaliłem kategorie II, III oraz IV w jedną, nadając jej wartość 1, co rozpatruje jako stan chory, a kategorie I zmieniłem na 0 i rozpatruje ją jako zdrowy. Ten zabieg pozwala na wskazanie pacjentów, którzy pilniej potrzebują opieki, dodatkowo algorytmy lepiej radzą sobie z problemem klasyfikacji binarnej.

☐ Eliminacja duplikatów mogących zafałszować wyniki

Aby uniknąć sytuacji, gdzie dany pacjent występuje w zbiorze więcej niż jeden raz, korzystam z funkcji pythona we współpracy z biblioteką pandas w celu usunięcia powtarzających się rekordów. Krok ten podjąłem, aby nie fałszować wyników w trakcie trenowania i oceny modeli.

☐ Identyfikacji i usuwaniu obserwacji odstających

W celu uniknięcia danych, które mogły zostać błędnie wprowadzone np. nieostrożność przy wpisywaniu wartości liczbowych (jeżeli wiek danego pacjenta wynosi 550, to można

uznać taką wartość za błędną, a tym samym za odstającą i fałszującą wyniki), więc szukam i usuwam wartości odstające. Do wartości odstających zaliczają się również błędy pomiarowe [19]. W celu identyfikacji wartości odstających wykorzystałem wykresy pudełkowe (ang. *box plots*), następnie podejmowałem decyzje czy je usunąć, czy zostawić, ponieważ usunięcie obserwacji odstających może prowadzić do utraty istotnych obserwacji, więc każdy przypadek rozpatrywałem indywidualnie.

❑ SMOTE

Zastosowałem metodę SMOTE (ang. *Synthetic Minority Over-sampling Technique*), która zwiększa w sposób syntetyczny i zrównoważony liczbę próbek dla klasy mniejszościowej. SMOTE generuje nowe próbki poprzez interpolacje istniejących obserwacji [20]. Według autorów artykułu jest to świetny sposób na zwiększenie liczby próbek niedostatecznie reprezentowanych. Więc w celu zbalansowania kategorii ‘chory’ oraz ‘zdrowy’ używam tej metody. Dzięki temu modele mają większą szansę nauczenia się cech charakterystycznych także dla mniej licznej kategorii, co przekłada się na lepsze rozpoznawanie przypadków chorobowych.

❑ Standaryzacja

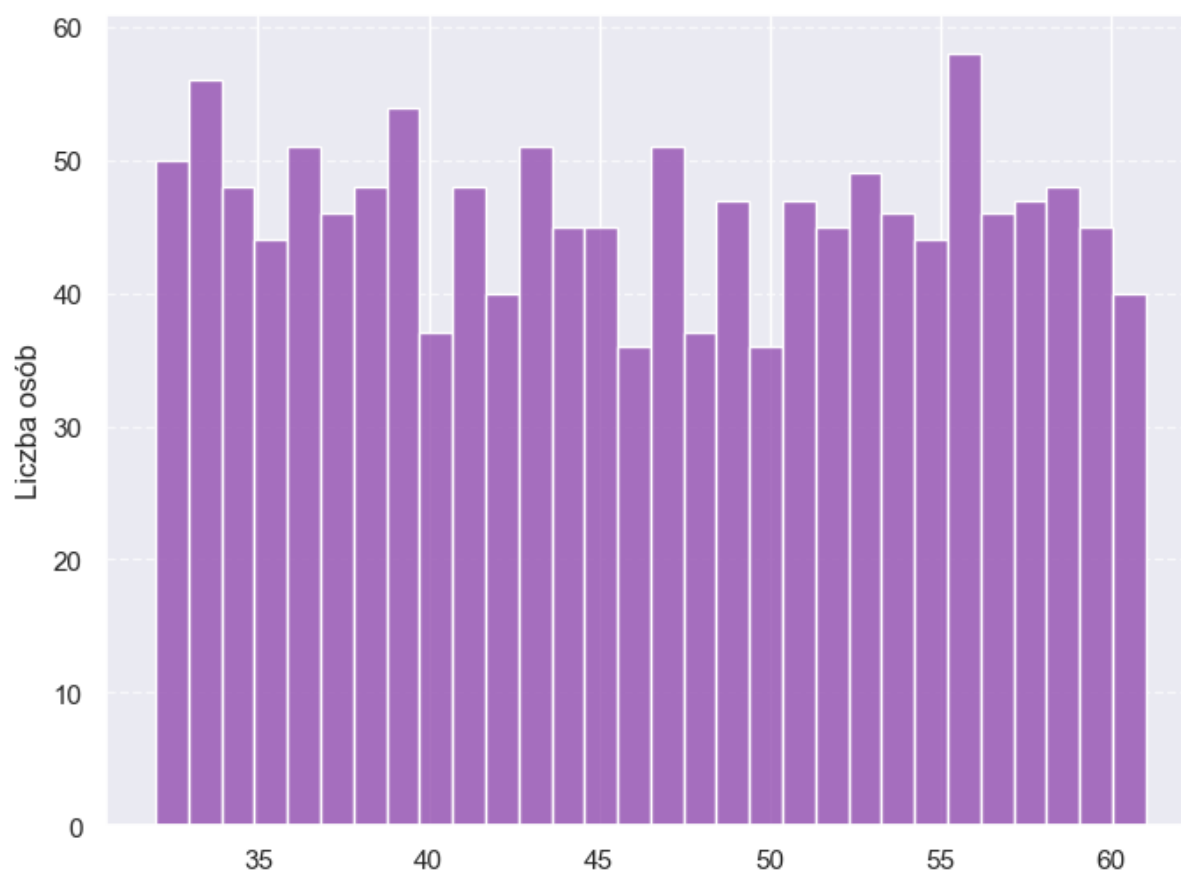
Dokonałem standaryzacji danych numerycznych, czyli zmodyfikowałem rozkład danej cechy w taki sposób, że średnia wartość wynosi 0, a odchylenie standardowe wynosi 1 [21]. Skorzystałem z biblioteki pythona scikit-learn i zaimplementowałem StandardScaler. Standaryzacji dokonałem jedynie na zbiorze treningowym, ze względów praktycznych, żeby zapewnić autentyczne działanie na danych, których model nie zna (zbiór testowy).

Każdy z tych kroków jest istotny w kontekście poprawnego wstępnego przetwarzania danych i rzetelności uzyskanych wyników. Po tych krokach zbiór jest spójny oraz zbalansowany, pozbawiony wartości odstających i duplikatów.

4.4 Analiza eksploracyjna danych

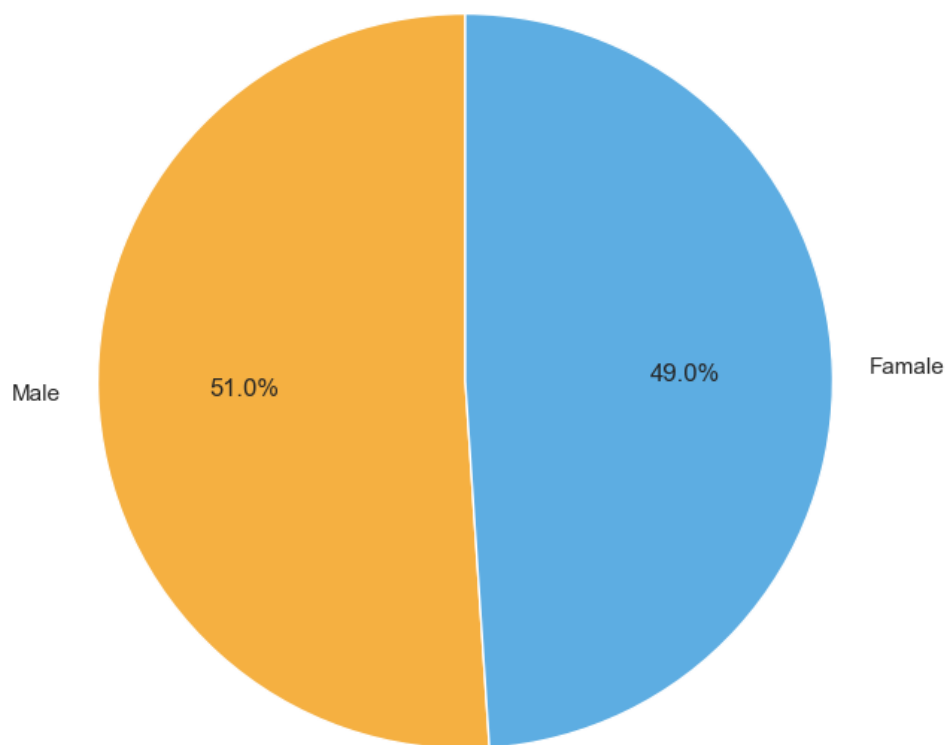
W tym podrozdziale przedstawiam liczne wykresy, aby w przystępny sposób zwizualizować rozkład cech oraz korelacje między danymi w zbiorze danych. W ten sposób umożliwiam odbiorcy lepsze zapoznanie się ze zbiorem. Wydaje mi się, że poprzez graficzny format przedstawienia danych, czytelnik łatwiej zrozumie kroki, które podjąłem w celu zbudowania skutecznego modelu sztucznej inteligencji przewidującego stany zwłóknienia wątroby, niż gdybym zapoznał go jedynie z surowymi danymi.

Rysunek [2] przedstawia histogram ilustrujący rozkład grup wiekowych pacjentów w analizowanym zbiorze danych. Można zauważyć, że najwięcej jest pacjentów w wieku 56 lat.



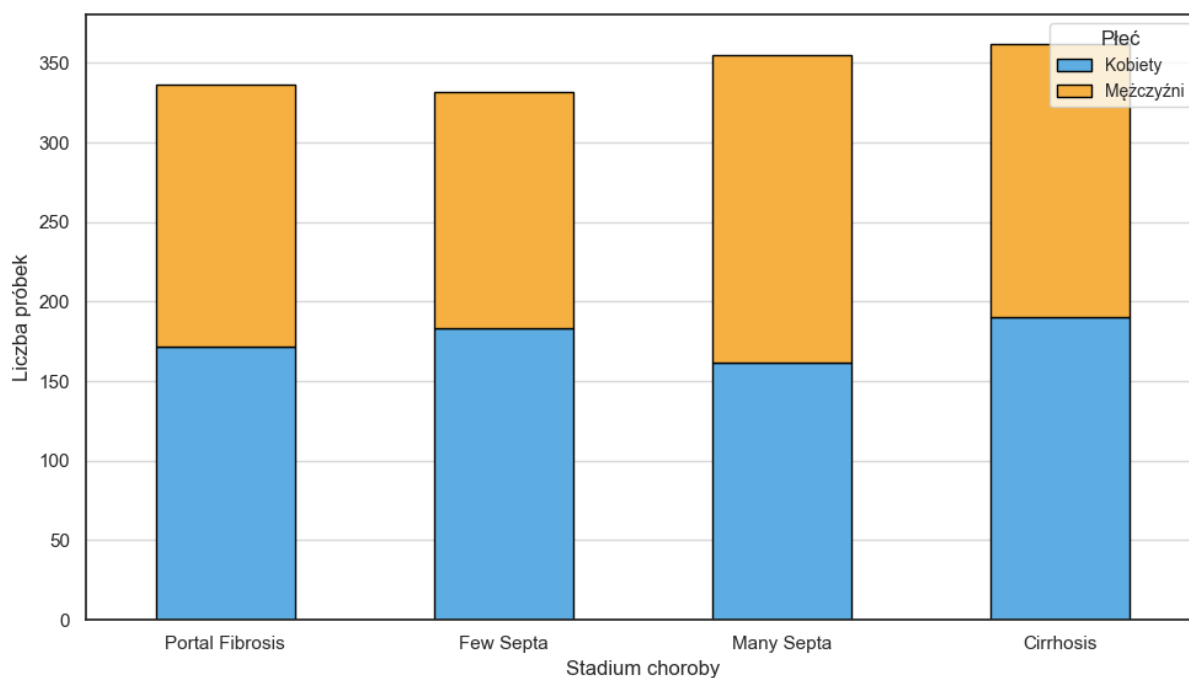
Rysunek 2 - histogram obrazujący grupy wiekowe pacjentów

Rysunek [3] przedstawia rozkład płci w badaniu. Można dostrzec, że zbiór danych jest zbalansowany pod tym względem, mimo niewielkiej przewagi liczebnej mężczyzn.



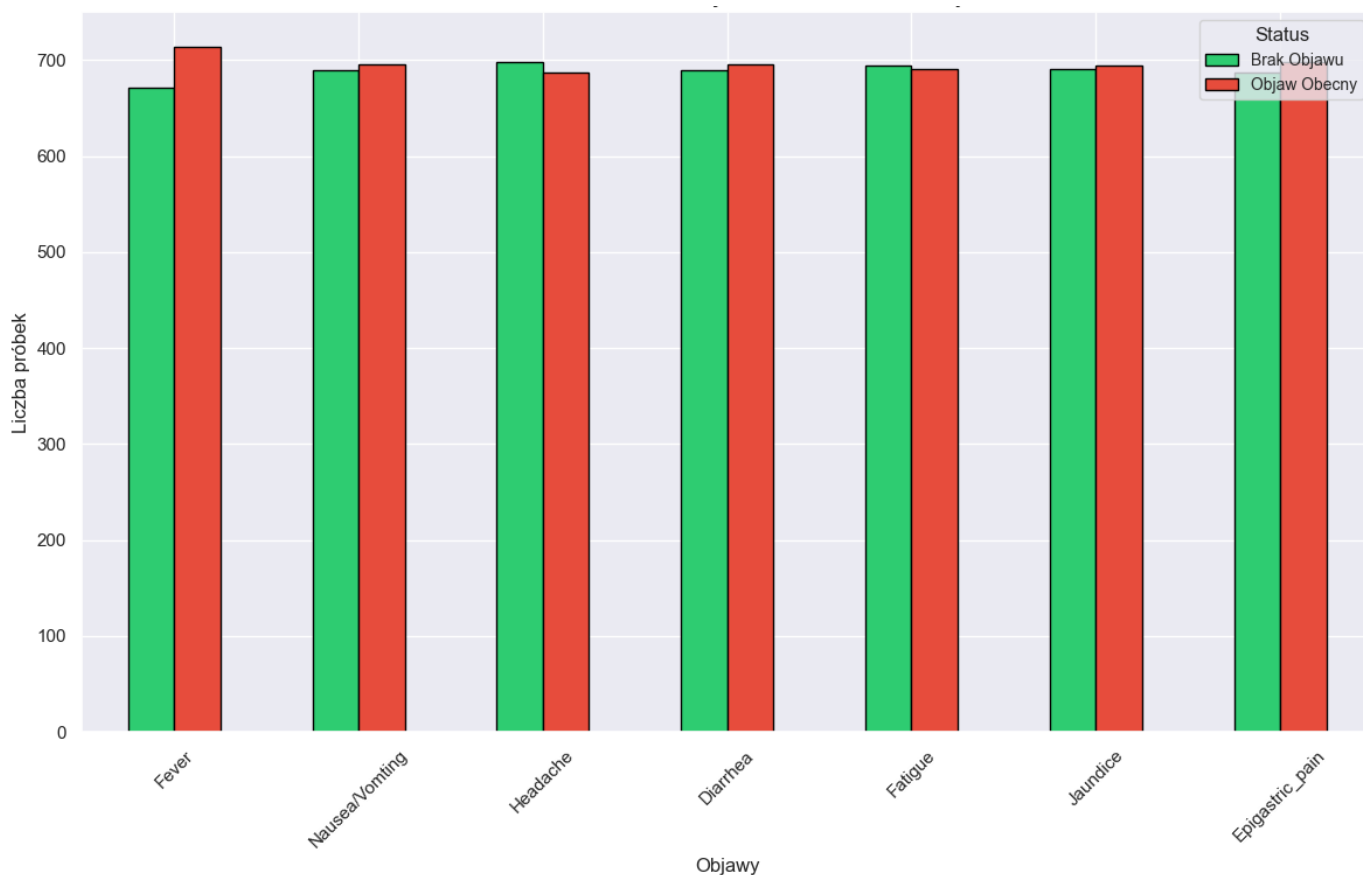
Rysunek 3 - wykres kołowy przedstawiający rozkład płci w badaniu

Rysunek [4] przedstawia udział mężczyzn i kobiet w różnych stadiach choroby. Wynika z niego, że w poszczególnych stadiach choroby nie ma znaczących odchyleń.



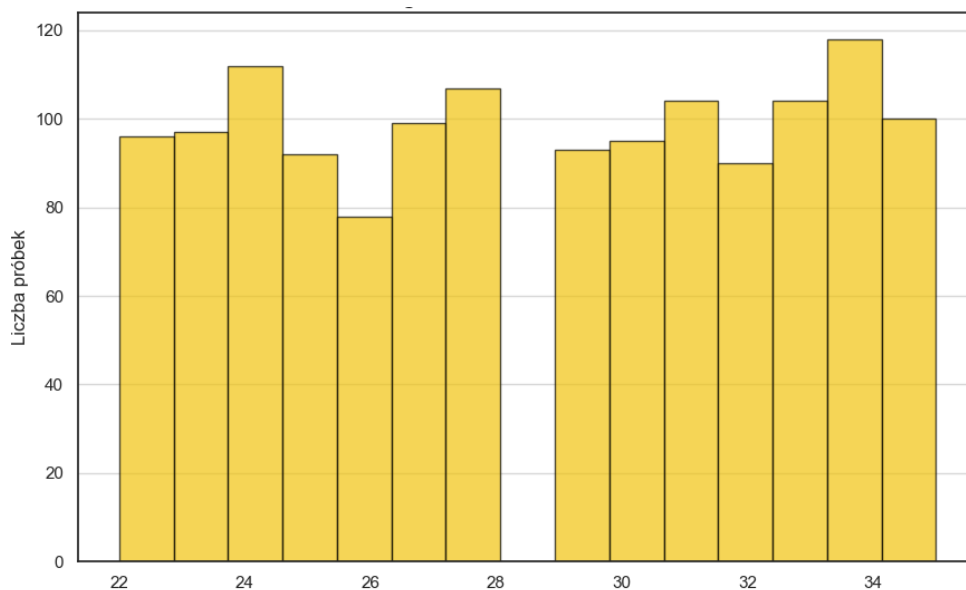
Rysunek 4 - wykres słupkowy obrazujący rozkład płci względem stadium choroby

Rysunek [5] przedstawia rozkład obecności objawów w zbiorze danych. Wynika z tego, że występowanie objawów jest zbalansowane, bez znaczących różnic.



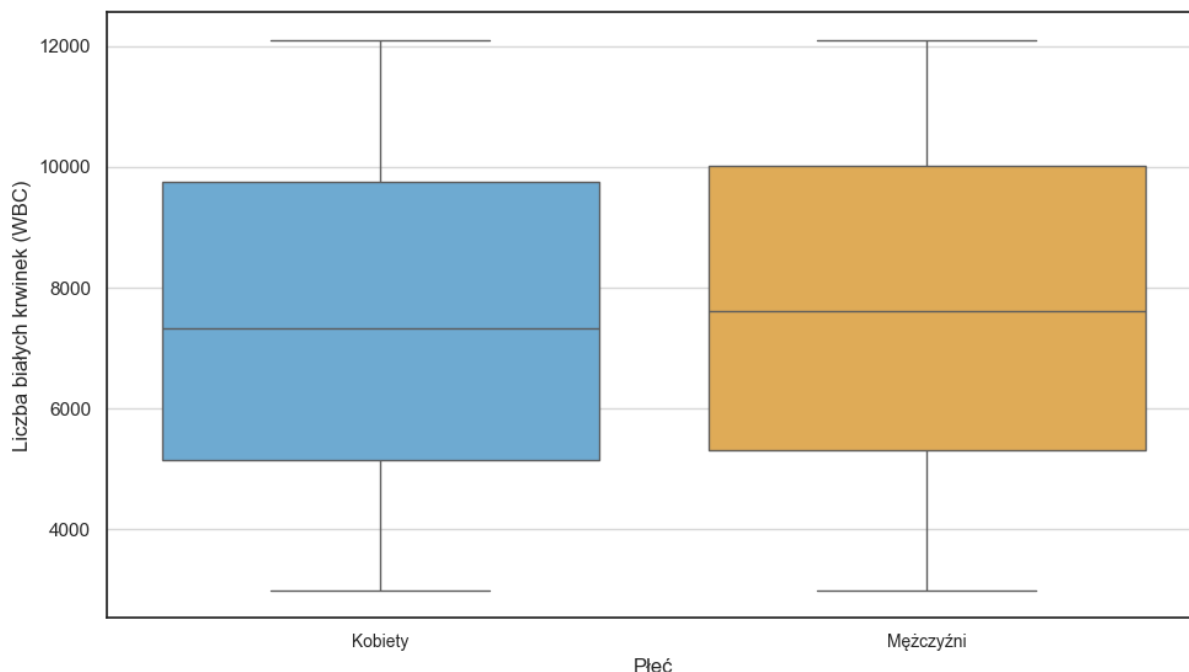
Rysunek 5 - wykres słupkowy przedstawiający rozkład objawów w badaniu

Rysunek [6] obrazuje rozkład wartości BMI wśród pacjentów. Wynika z niego, że zakres wartości BMI to od 22 do 35.



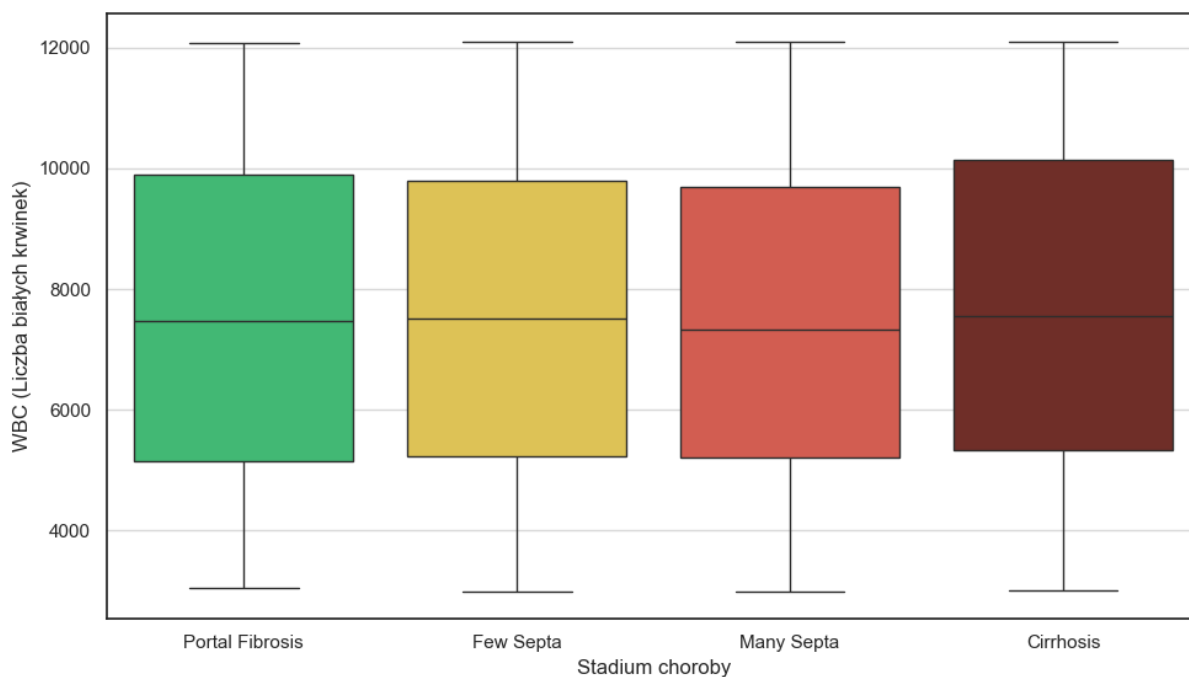
Rysunek 6 - histogram przedstawiający rozkład BMI w badaniu

Rysunek [7] przedstawia rozkład wartości WBC względem płci. Wynika z niego, że mediana liczby białych krwinek u mężczyzn jest nieznacznie większa niż u kobiet.



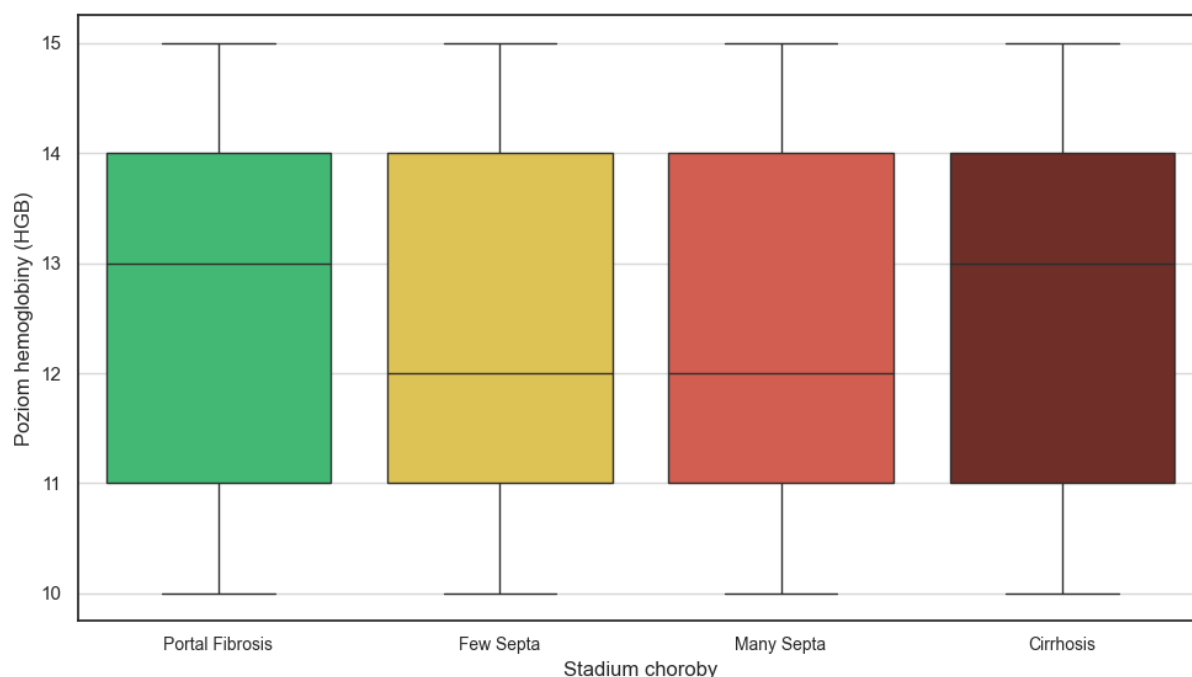
Rysunek 7 - wykres pudełkowy obrazujący rozkład WBC względem płci w badaniu

Rysunek [8] przedstawia rozkład WBC w zależności od stadium choroby. Można zauważyć, że mediana liczby białych krwinek nieznacznie się różni w zależności od stadium choroby.



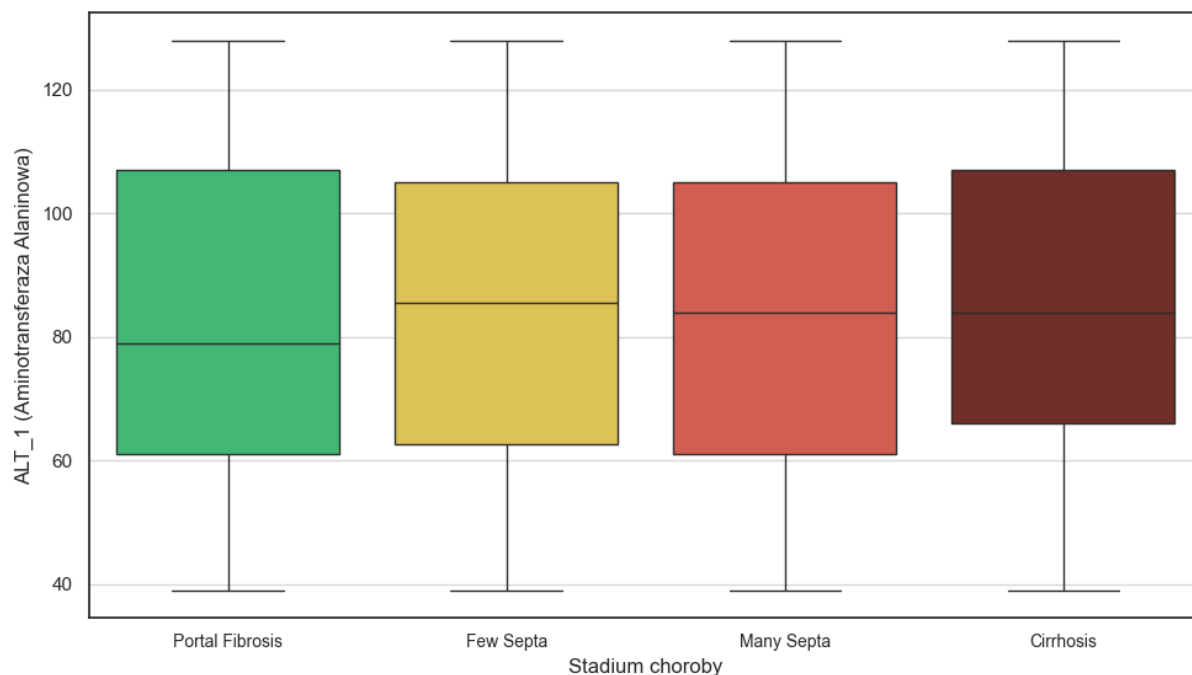
Rysunek 8 - wykres pudełkowy obrazujący rozkład WBC w zależności od stadium choroby

Rysunek [9] przedstawia rozkład poziomu hemoglobiny względem stadiów choroby. Wynika z niego, że u osób we wczesnym stadium oraz z marskością wątroby poziom hemoglobiny jest znacząco wyższy niż u osób w postępujących stadiach.



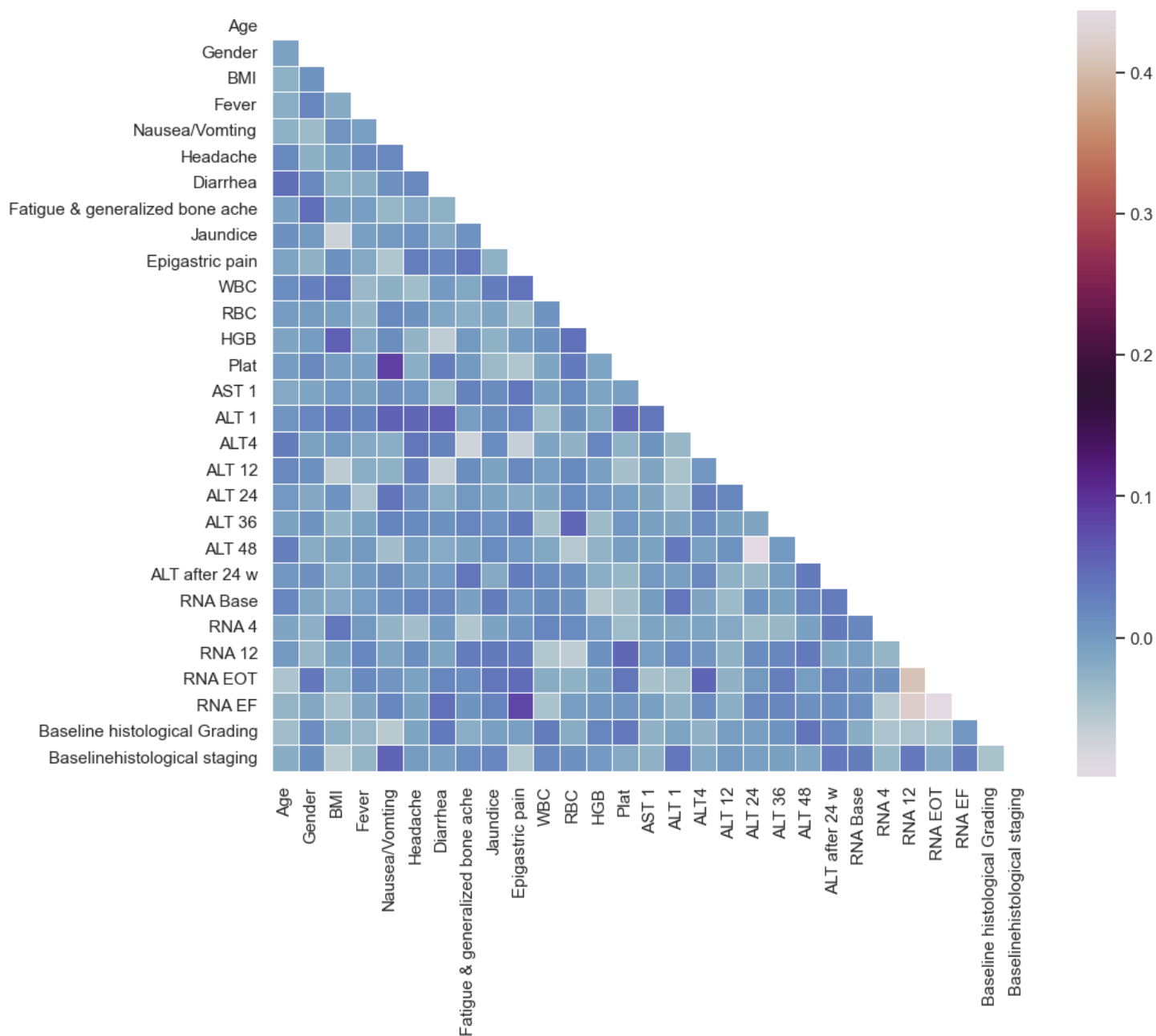
Rysunek 9 - wykres obrazujący rozkład poziomu hemoglobiny względem stadiów choroby

Rysunek [10] przedstawia rozkład enzymu ALT w różnych stadiach choroby. Wynika z niego, że u osób w pierwszym stadium poziom tego enzymu jest niższy niż w pozostałych.



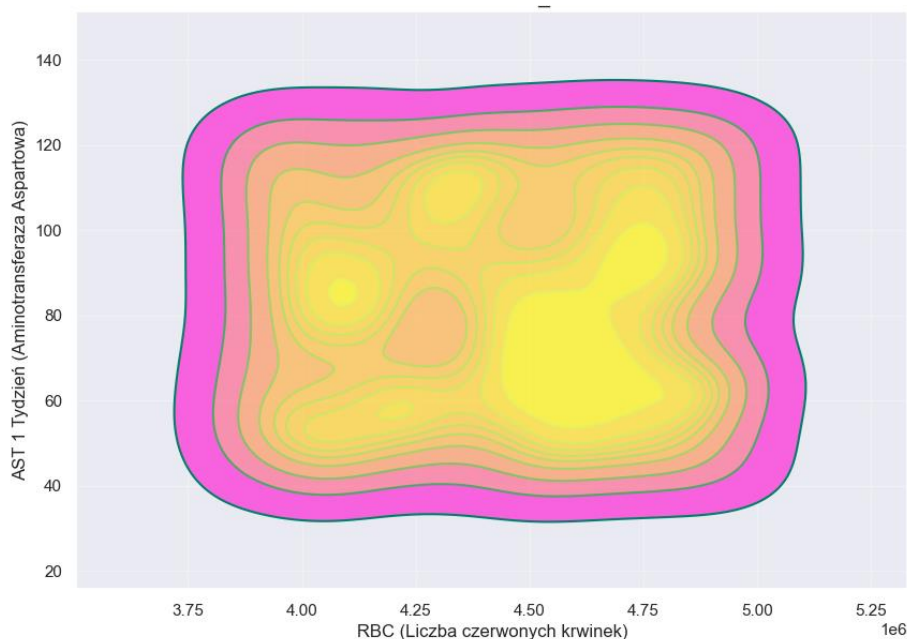
Rysunek 10 - wykres obrazujący rozkład enzymu ALT w różnych stadiach choroby

Rysunek [11] przedstawia korelacje między zmiennymi numerycznymi w zbiorze danych. Wynika z niego, że poziom RNA na końcu przebiegu leczenia jest silnie skorelowany z poziomem RNA w dwunastym tygodniu od rozpoczęcia badania.



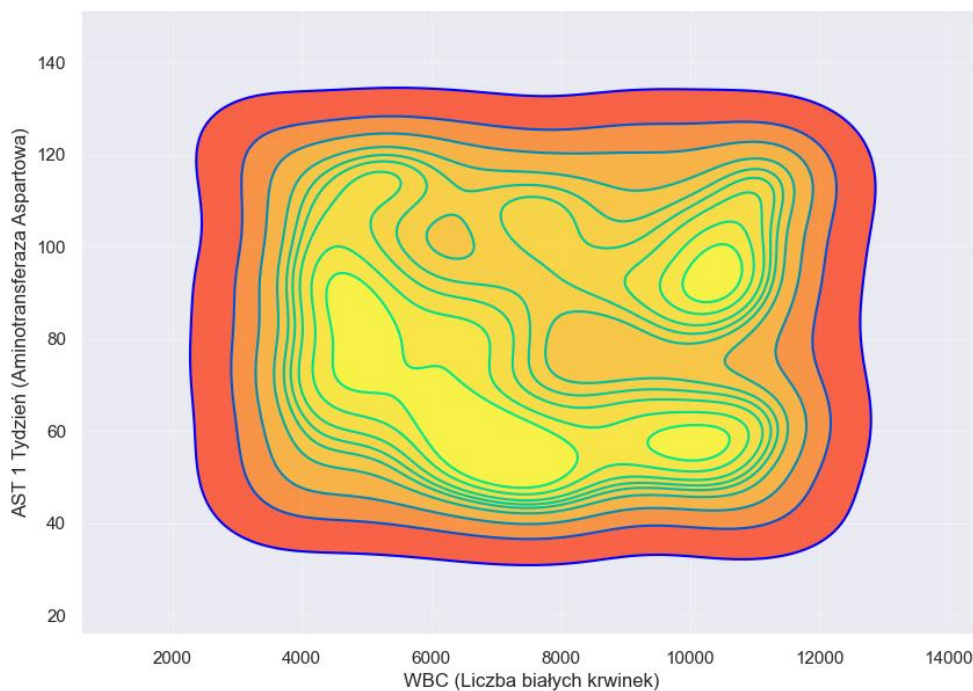
Rysunek 11 - wykres obrazujący skorelowanie cech numerycznych w zbiorze danych

Rysunek [12] przedstawia zagęszczenie danych dla liczby czerwonych krwinek oraz enzymu AST. Wykres pozwala zaobserwować, które zakresy wartości liczby czerwonych krwinek są bardziej związane z podwyższonym poziomem enzymu AST.



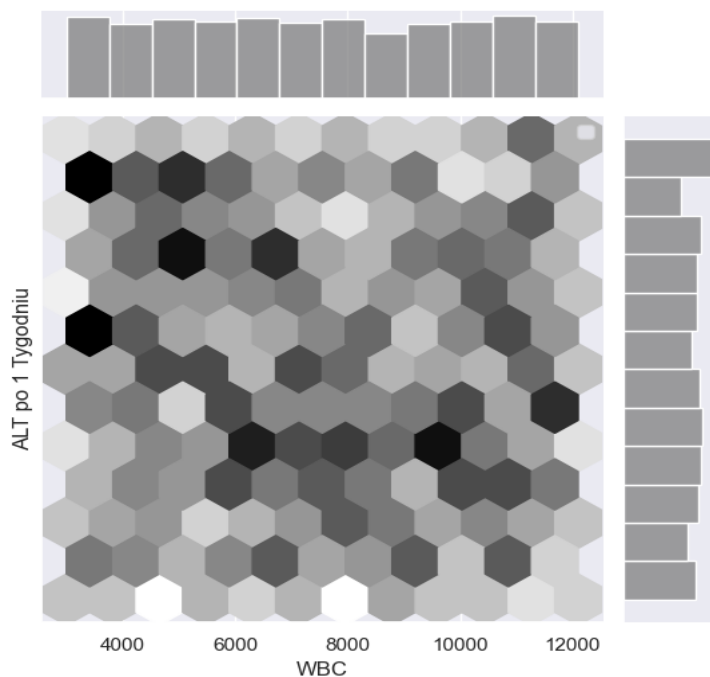
Rysunek 12 - heatmapa zależności między liczbą czerwonych krwinek a poziomem enzymu AST

Rysunek [13] przedstawia zagęszczenie danych dla liczby białych krwinek oraz enzymu AST. Z wykresu łatwo zaobserwować, które zakresy wartości liczby białych krwinek są bardziej skorelowane z podwyższonym poziomem enzymu AST.

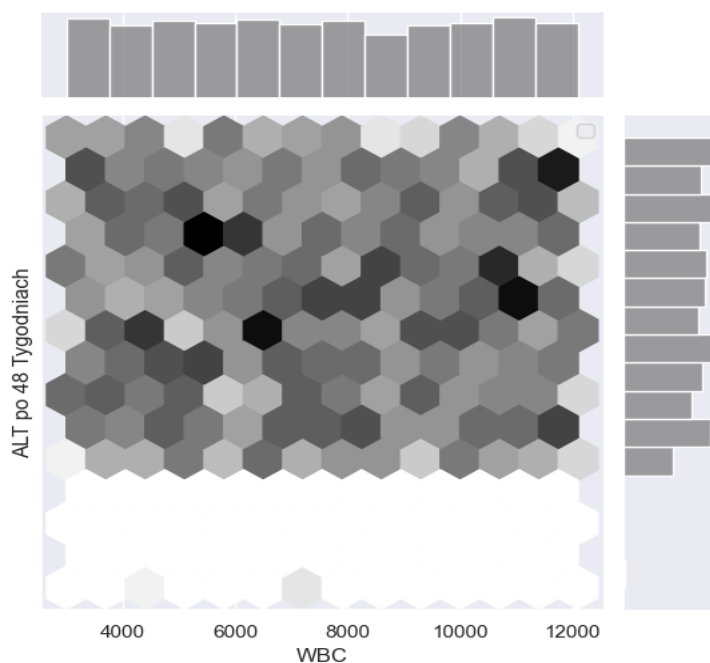


Rysunek 13 - heatmapa zależności między liczbą białych krwinek a poziomem enzymu AST

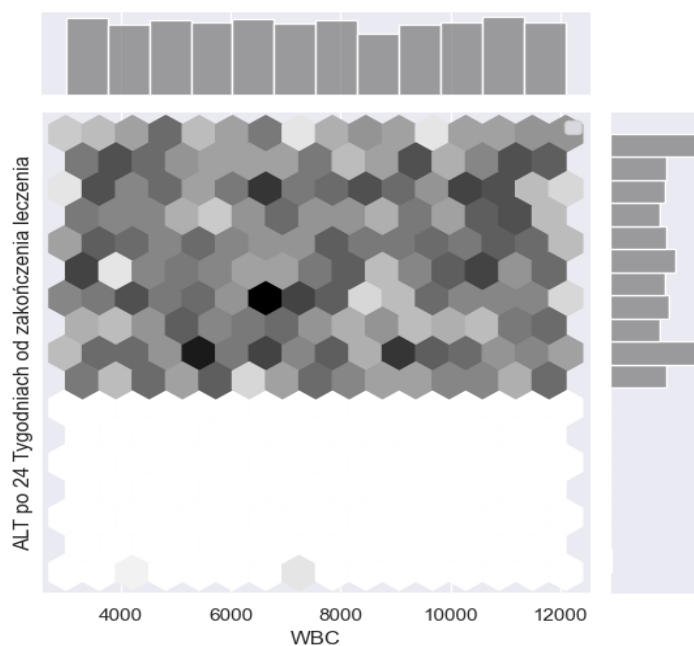
Rysunki [14, 15, 16] przedstawiają zależność między poziomem ALT (w różnych tygodniach od rozpoczęcia leczenia), a liczbą białych krwinek. Te wykresy heksagonalne wizualizują zagęszczenie danych, pomagając zidentyfikować, czy zmiany w poziomie ALT korelują z różnymi wartościami WBC.



Rysunek 14 - wykres heksagonalny obrazujący skorelowanie WBC z enzymem ALT w pierwszym tygodniu od rozpoczęcia leczenia

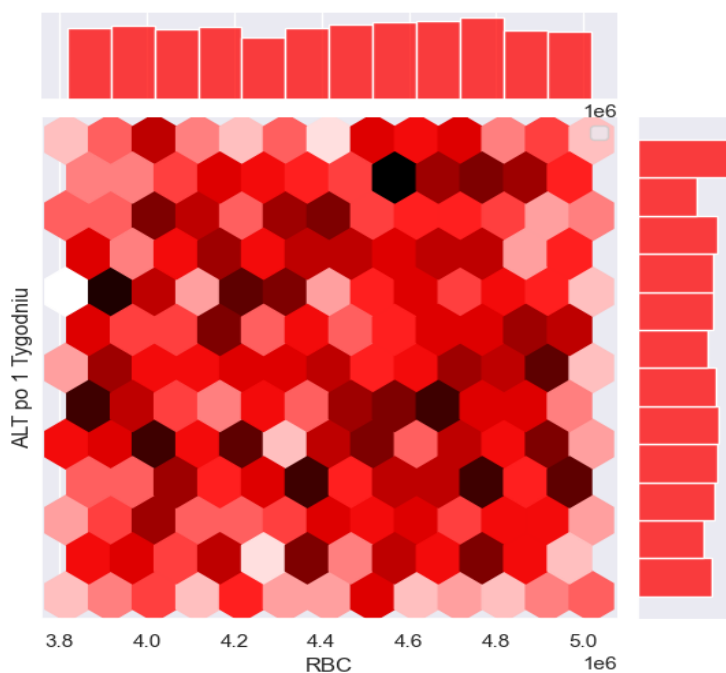


Rysunek 15 - wykres heksagonalny obrazujący skorelowanie WBC z enzymem ALT po 48 tygodniach od rozpoczęcia badania

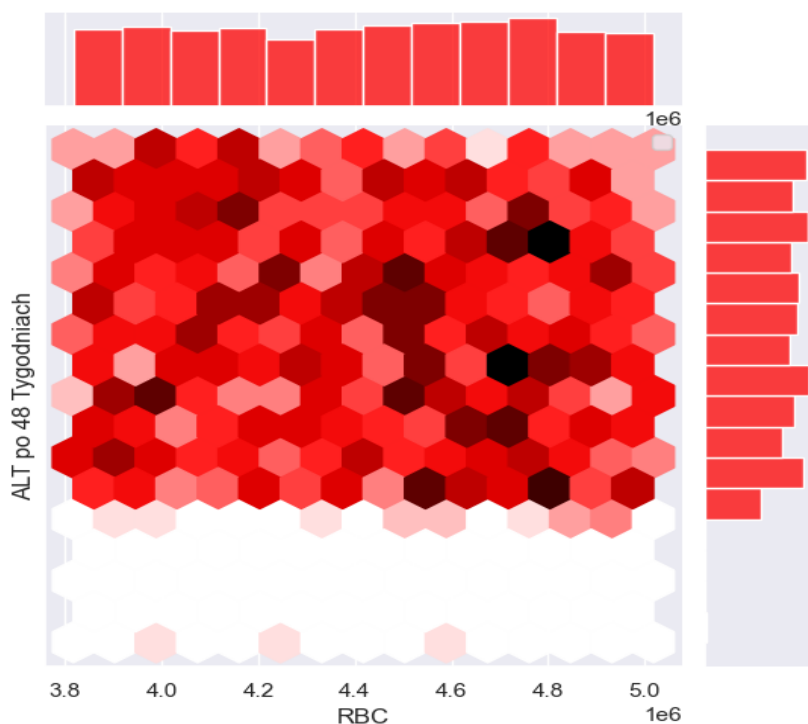


Rysunek 16 - wykres heksagonalny obrazujący skorelowanie WBC z enzymem ALT po 24 tygodniach od zakończenia leczenia

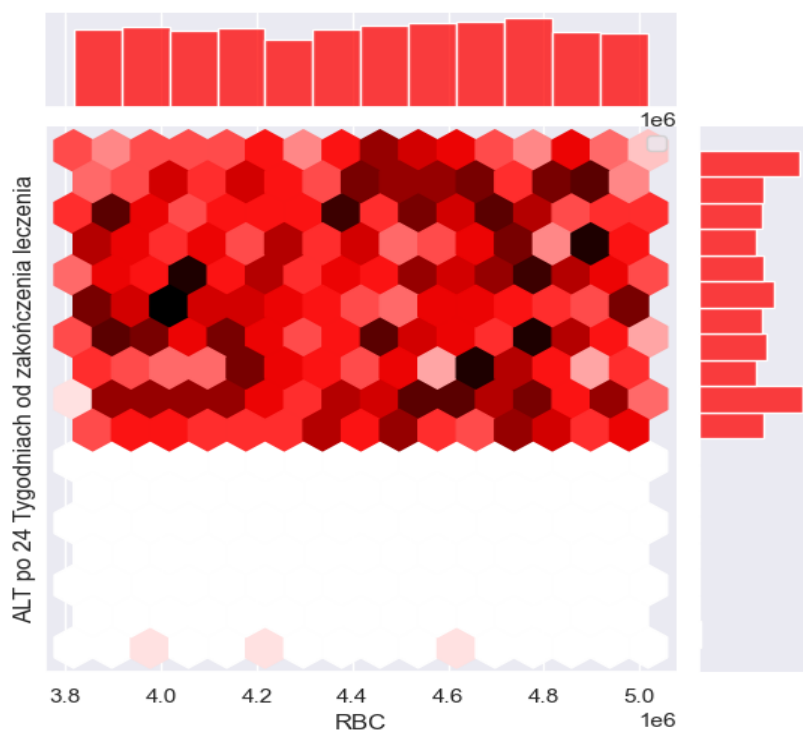
Rysunki [17, 18, 19] przedstawiają zależność między poziomem ALT (w różnych tygodniach od rozpoczęcia leczenia), a liczbą czerwonych krwinek. Te wykresy heksagonalne wizualizują zagęszczenie danych, pomagając zidentyfikować, czy zmiany w poziomie ALT korelują z różnymi wartościami RBC.



Rysunek 17 - wykres heksagonalny obrazujący skorelowanie RBC z enzymem ALT w pierwszym tygodniu od rozpoczęcia leczenia

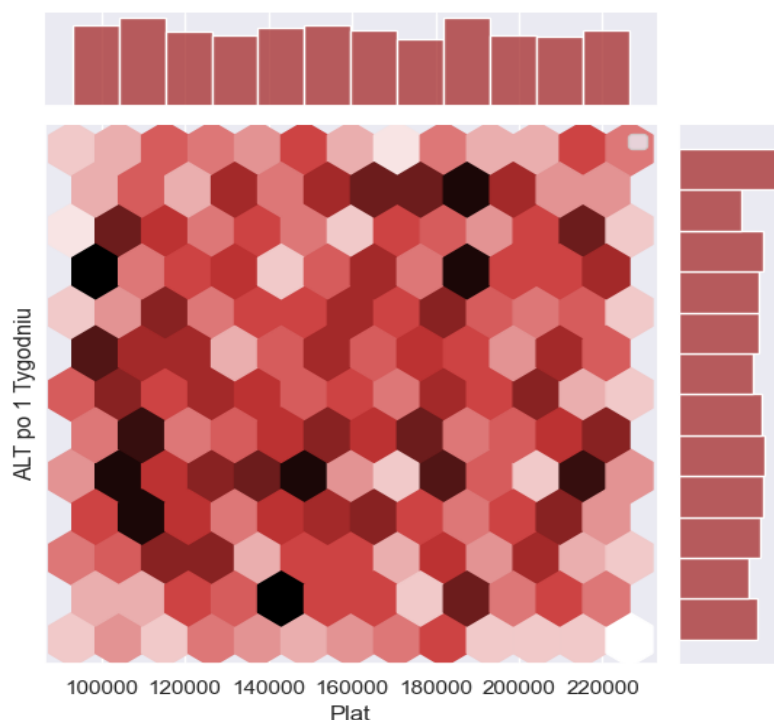


Rysunek 18 - wykres heksagonalny obrazujący skorelowanie RBC z enzymem ALT po 48 tygodniach od rozpoczęcia leczenia

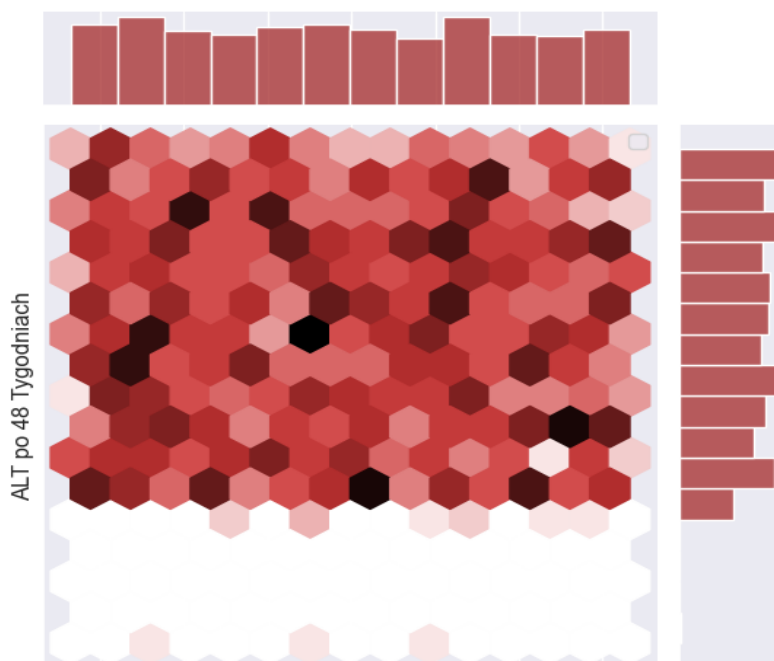


Rysunek 19 - wykres heksagonalny obrazujący skorelowanie RBC z enzymem ALT po 24 tygodniach od zakończenia leczenia

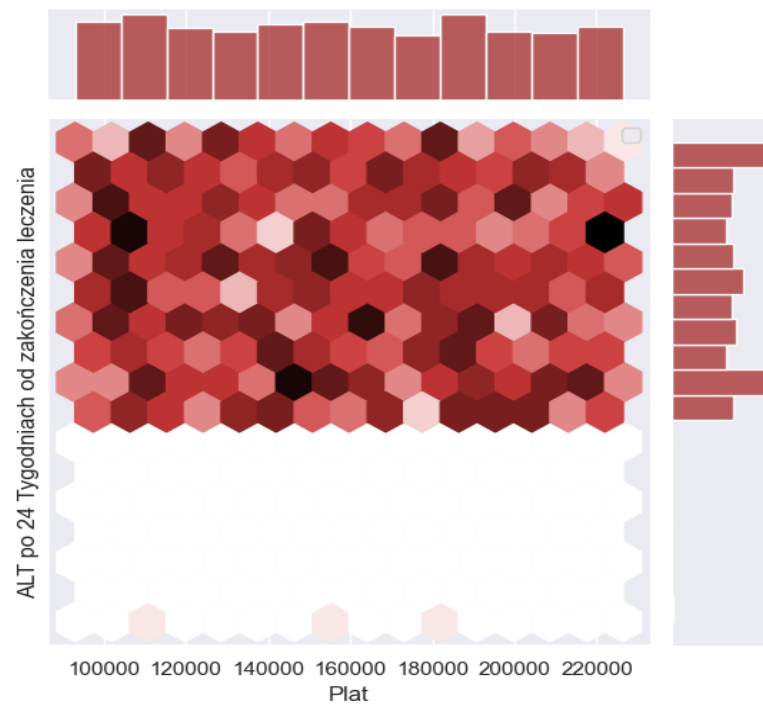
Rysunki [20, 21, 22] przedstawiają zależność między poziomem ALT (w różnych tygodniach od rozpoczęcia leczenia), a liczbą płytek krwi. Te wykresy heksagonalne wizualizują zagęszczenie danych, pomagając zidentyfikować, czy zmiany w poziomie ALT korelują z różnymi wartościami liczby płytek krwi.



Rysunek 21 - wykres heksagonalny obrazujący skorelowanie liczb płytek krwi z enzymem ALT po pierwszym tygodniu od rozpoczęcia leczenia



Rysunek 20 - wykres heksagonalny obrazujący skorelowanie liczby płytek krwi z enzymem ALT po 48 tygodniach od rozpoczęcia leczenia



Rysunek 22 - wykres heksagonalny obrazujący skorelowanie liczby płytek krwi z enzymem ALT po 24 tygodniach od zakończenia leczenia

5. Budowa modeli klasyfikacyjnych

5.1 Wybrane algorytmy

W niniejszej pracy wybrałem dziesięć algorytmów uczenia maszynowego, które zostały zoptymalizowane za pomocą dwóch technik optymalizacyjnych dając w rezultacie trzydzieści modeli predykcyjnych, dla każdego algorytmu zbudowałem model bez optymalizacji parametrów, model optymalizowany za pomocą random search oraz model optymalizowany za pomocą optuny. Poniższe algorytmy różnią się od siebie podejściem do analizy danych, dzięki czemu mogę porównać ich efektywność w przewidywaniu stadiów wątroby [22-25]. Dokonałem krótkiej charakterystyki każdego zaimplementowanego algorytmu:

❑ Las losowy (ang. *Random Forest*)

Las losowy to algorytm zaproponowany przez Leo Breimana, opierający się grupie drzew decyzyjnych. Każde drzewo podejmuje decyzje na podstawie losowego podzbioru danych i cech. Ostateczny wynik klasyfikacji uzyskiwany jest na drodze głosowania większościowego. Wybrałem ten algorytm, ponieważ jest odporny na przeuczenie (ang. *overfitting*) i brak danych, dodatkowo jest zdolny do uchwycenia złożonych zależności.

❑ XGBoost (ang. *Extreme Gradient Boosting*)

XGBoost to zaawansowany algorytm uczenia maszynowego korzystający z koncepcji drzew decyzyjnych (ang. *Decision tree*) i wzmocnienia gradientowego (ang. *Gradient boosting*). Wzmocnienie gradientowe to koncepcja, gdzie każdy model jest budowany iteracyjnie i starający się poprawić błędy poprzedniego modelu. Wybrałem ten algorytm, ponieważ cechuje się dobrą wydajnością przy średnich zbiorach danych oraz jest jednym z najczęściej wybieranych algorytmów w przemyśle i badaniach.

❑ LightGBM (ang. *Light gradient-boosting machine*)

Algorytm LightGBM oparty jest na koncepcji wzmocnienia gradientowego oraz drzewach decyzyjnych. Wykorzystuje również koncepcje podziału na podstawie liści (ang. *Leaf-wise splitting*), tym samym zapewniając redukcję strat oraz osiąganie większej dokładności. Zdecydowałem się wybrać ten algorytm, ponieważ wyróżnia się szybkością oraz mniejszym zużyciem pamięci. Często jest wykorzystywany w zadaniach związanych z klasyfikacją.

❑ Maszyna wektorów nośnych (ang. *SVM – Support Vector Machine*)

SVM to nadzorowany algorytm uczenia maszynowego, który bardzo dobrze nadaje się do zadań klasyfikacji. Algorytm maksymalizuje margines między klasami, przez co skuteczniej rozdziela dane. Kluczowe punkty to wektory nośne (ang. *Support vectors*), które definiują granice decyzyjne. Wybrałem ten algorytm, ponieważ cechuje się odpornością na dopasowanie oraz wszechstronnością.

❑ Regresja logistyczna (ang. *Logistic Regression*)

Algorytm regresji logistycznej służy do klasyfikacji binarnej, klasyfikuje poprzez znalezienie optymalnej linii separującej klasy w przestrzeni wielowymiarowej. Uczenie algorytmu polega na minimalizacji funkcji błędu. Regresja logistyczna jest podstawową metodą klasyfikacji, dlatego nie mogło jej zabraknąć w mojej pracy. Zdecydowałem się na jej wybór z uwagi na przystosowanie do klasyfikacji binarnej.

❑ K-najbliższych sąsiadów (ang. *KNN – K-nearest neighbors*)

Algorytm KNN jest zaliczany do grupy algorytmów leniwych (ang. *lazy*), co znaczy, że w odróżnieniu od SVM, gdzie model jest tworzony w procesie uczenia, KNN działa bezpośrednio na danych uczących. Każda nowa klasyfikacja wymaga analizy całego zbioru danych treningowych od początku. Zdecydowałem się na wybór tego algorytmu, ponieważ radzi sobie z problemami klasyfikacji oraz jest łatwy w implementacji.

❑ Naiwny klasyfikator Bayesa (ang. *Gaussian Naive Bayes*)

Naiwny klasyfikator Bayesa to algorytm uczenia maszynowego przeznaczony do klasyfikacji. Model ten opiera się na teorii prawdopodobieństwa Bayesa, maksymalizując prawdopodobieństwo warunkowe przypisania obiektu do danej klasy. W celu osiągnięcia lepszych rezultatów dobrze, gdy zbiór uczący jest odpowiednio duży. Algorytm wydajny obliczeniowo, lepiej działający na dużych zbiorach danych. Wybrałem go, aby porównać jego wyniki z algorytmami, które lepiej radzą sobie na mniejszych danych.

❑ Drzewo decyzyjne (ang. *Decision Tree*)

Drzewo decyzyjne to algorytm uczenia maszynowego stosowany do klasyfikacji (może być stosowany również do regresji). Struktura drzewa decyzyjnego wygląda jak podejmowanie decyzji, gdzie obserwacje są klasyfikowane na podstawie pytań reprezentowanych przez węzły (ang. *Nodes*) i liście (ang. *Leafs*), każdy węzeł reprezentuje atrybut, a każdy liść reprezentuje rezultat. Algorytm drzewa decyzyjnego został rozszerzony w wyniku czego powstał las losowy oraz boosting. Zastosowałem ten algorytm z uwagi na prostotę i łatwość implementacji.

❑ Sieć neuronowa (ang. *MLPClassifier*)

Sieć neuronowa to model uczenia maszynowego, łączący dane wejściowe, czyli atrybuty z danymi wyjściowymi, czyli zmienną przewidywaną. Istotny był wybór liczby warstw ukrytych (ang. *Hidden layers*), które zawierają neurony (ang. *neurons*) odpowiedzialne za obliczenia oraz przekształcenie danych wejściowych, co pozwala na uchwycenie relacji w zbiorze. Zaimplementowałem wielowarstwową perceptronową (ang. *perceptron*) sieć neuronową, ponieważ sprawdza się w skomplikowanych problemach klasyfikacyjnych.

❑ Maszyna wektorów nośnych z parametrem nu (ang. *NuSVM – nu-support vector machine*)

NuSVM to wariant klasyfikatora SVM, gdzie zamiast parametru C wykorzystuje parametr nu. Parametr ten określa dolną granicę ułamka wektorów nośnych, co przekłada się na większą kontrolę nad modelem. W wyniku czego NuSVM jest bardziej odporny na wartości odstające, lepiej radzi sobie z danymi zaszumionymi oraz pozwala na większą elastyczność w dostosowywaniu modelu. Zaimplementowałem ten wariant klasyfikatora, aby sprawdzić, czy przyniesie lepsze rezultaty niż jego podstawowa wersja SVM.

5.2 Optymalizacja modeli

Ważnym etapem mojej pracy jest optymalizacja modeli uczenia maszynowego, ponieważ właściwy dobór parametrów znacząco wpływa na efektywność predykcji. Skupiłem się na dwóch różnych technikach optymalizacji: *RandomizedSearchCV* oraz *Optuna*. Obie te techniki pozwoliły mi na znalezienie optymalnej konfiguracji każdego modelu [26].

❑ Metoda losowania (ang. *Random Search*)

Random search jest to technika optymalizacji hiperparametrów, gdzie losowo próbowane są różne kombinacje parametrów z określonego zakresu. Warto zastanowić się nad jej

wyborem, gdy liczba możliwych kombinacji jest bardzo duża. Niestety ta technika nie korzysta z informacji, z poprzednich wyników. Jako argument przyjmuje słownik z parametrami. Celem tej techniki nie jest znalezienie optymalnego rozwiązania (i tego nie gwarantuje), ale znalezienie dobrego rozwiązania w stosunkowo krótkim czasie. Zdecydowałem się na tą technikę dlatego, że oferuje losowość, co przekłada się na szansę znalezienia obiecujących ustawień w relatywnie krótkim czasie oraz dlatego, że nie jest wymagająca pod względem zasobów obliczeniowych.

❑ Optuna (ang. *Optuna*)

Optuna jest to zaawansowana technika optymalizacji hiperparametrów. W odróżnieniu od random search optuna korzysta z historii prób, aby określić, które wartości hiperparametrów dobrze byłoby wypróbować jako następne. Optuna poprzez metodę prób i błędów wyszukuje i znajduje optymalną konfigurację hiperparametrów. Jest to technika, która szczyci się doskonałą wydajnością. Optuna poszukuje obiecującego obszaru i stara się wybrać najlepsze wartości hiperparametrów, następnie szuka skrupulatniej obiecującego obszaru na podstawie wyników z poprzednich prób. Dla algorytmów takich jak XGBoost czy LightGBM jest szczególnie efektywna, ponieważ te modele mają wiele parametrów do dostrojenia.

Zdecydowałem się na wybór tych dwóch technik dlatego, że pozwoliły mi na dostrojenie hiperparametrów dla każdego modelu i uzyskanie najlepszych rezultatów. Są to dwie różne techniki, optuna jest bardziej wymagająca czasowo, ale teoretycznie powinna uzyskać lepsze wyniki niż random search, który ma za zadanie znaleźć dobre rozwiązanie w relatywnie krótkim czasie i przy niedużym obciążeniu sprzętowym.

5.3 Implementacja

Python (w wersji 3.12.7) to język programowania, który posłużył mi do przeprowadzenia wstępnego przetwarzania danych oraz implementacji konkretnych algorytmów. Zdecydowałem się na wybór tego języka, ponieważ python ma wiele bibliotek, które wspomagają pracę związane ze sztuczną inteligencją, prostą składnię oraz zauważyłem, że język ten cieszy się dużą popularnością wśród osób zajmujących się analizą danych i uczeniem maszynowym. Visual Studio Code jest to moje ulubione zintegrowane środowisko programistyczne (ang. *IDE – Integrated Development Enviroment*) z licznymi motywami graficznymi i rozszerzeniami np. Notatnikiem Jupyter (ang. *Jupyter Notebook*). Notatnik Jupyter jest dosyć często wykorzystywany przez osoby, które piszą artykuły na temat sztucznej inteligencji, więc postanowiłem z niego skorzystać. Osobiście podoba mi się to rozwiązanie, ponieważ oprócz pisania kodu w Pythonie, notatnik Jupyter obsługuje Markdown, dzięki czemu prowadzenie dokumentacji bezpośrednio w notatniku jest proste i wygodne. Dodatkowo możliwość testowania fragmentów kodu urzekła mnie na tyle bym skorzystał z tego rozwiązania.

W tym podrozdziale skupiam się na implementacji algorytmu regresji logistycznej, budując trzy modele. Niemniej jednak dla innych algorytmów proces implementacji wygląda dosyć podobnie.

Kod dla wstępnego przetwarzania danych, wizualizacji danych oraz implementacji poszczególnych algorytmów wraz z dokumentacją znajduje się w moim repozytorium na GitHub: [Odnóśnik do repozytorium GitHub](#).

Rysunek [23] przedstawia zaimportowane biblioteki, które są niezbędne do sprawniej budowy modeli oraz wizualizacji ich wyników. Są to takie biblioteki jak pandas, służąca do obsługi danych tabelarycznych, numpy wspierająca wykonywanie operacji numerycznych, optuna służąca do optymalizacji modelu za pomocą optuny, matplotlib oraz seaborn umożliwiające wizualizacje wyników modeli oraz sklearn skąd importuje modele w wersji podstawowej, metryki do oceny modelu oraz RandomizedSearchCV, czyli gotowe podejście koncepcji random search.

```
# =====  
#         importy  
# =====  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import roc_curve, roc_auc_score, accuracy_score, precision_score, recall_score, confusion_matrix  
from sklearn.model_selection import RandomizedSearchCV  
import optuna
```

Rysunek 23 - fragment kodu implementacji regresji logistycznej – importowane biblioteki

Rysunek [24] obrazuje budowanie modelu regresji logistycznej bez optymalizacji. Najpierw inicjalizuje gotowy model regresji logistycznej, z pomocą przychodzi biblioteka sklearn, parametr random_state zapewnia, że wyniki przy każdym uruchomieniu kodu będą takie same, a max_iter mówi o maksymalnej liczbie iteracji algorytmu w procesie treningu modelu.

Następnie trenuje model za pomocą metody fit() na danych treningowych, X_train to zbiór treningowy zawierający cechy, a y_train zawiera jedynie etykiety klasy docelowej. W tym kroku model ustala wagi (ang. weights) dla atrybutów oraz minimalizuje funkcję kosztu (ang. cost function). Wagi są to liczby przypisane przez model do danej cechy, informują o tym jak dana cecha jest ważna w procesie predykcji. Z kolei funkcja kosztu dostarcza informacji jak bardzo przewidywania modelu różnią się od rzeczywistych wartości.

Idąc dalej zbiór X_test zawiera dane, na podstawie których model ma przewidzieć etykietę docelową (baseline histological staging), wynik predykcji zapisuje w wektorze y_pred_no_optimization. Z kolei metoda predict_proba zwraca macierz prawdopodobieństw przynależności danej próbki w X_test do poszczególnych klas, posłuży mi to do oceny wyników modeli. Na koniec otrzymuje y_pred_no_optimization gdzie są zapisane predykcje modelu na podstawie danych testowych oraz probs_no_optimization gdzie gromadzę prawdopodobieństwa przypisania klasy pozytywnej (zapalenie wątroby) przez model.

```
# =====  
#         bez optymalizacji  
# =====  
model_no_optimization = LogisticRegression(random_state=42, max_iter=10000)  
model_no_optimization.fit(X_train, y_train)  
y_pred_no_optimization = model_no_optimization.predict(X_test)  
probs_no_optimization = model_no_optimization.predict_proba(X_test)[: , 1]
```

Rysunek 24 - fragment kodu implementacji regresji logistycznej - model nie optymalizowany

Rysunek [25] to fragment kodu implementacji modelu wykorzystujący optune do optymalizacji hiperparametrów. Najpierw tworze funkcje optuna_optimization(), celem tej funkcji jest dobrać losową wartość dla parametru C od 0.0001 do 10000 (im mniejsze C tym większa kara za duże wagi), zainicjować instancje regresji logistycznej z wylosowaną wartością

hiperparametru C stosując regularyzację L2 (ang. *L2 Regularization*) oraz lbfgs (ang. *Limited-memory Broyden-Fletcher-Goldfarb-Shanno*), następnie wytrenować model oraz ocenić jego skuteczność na podstawie metryki AUC. Każde wywołanie funkcji to jedna iteracja optymalizacji. Regularyzacja L2 stosuje, aby uniknąć przetrenowania, w tym celu regularyzacja L2 dodaje kary do funkcji kosztu wymuszając tym samym zmniejszenie wartości wag przez model. Algorytm lbfgs to technika służąca do minimalizacji funkcji kosztu.

Potem tworze instancje obiektu `optuna` w celu przeprowadzenia optymalizacji (`create_study()`), moim celem jest maksymalizacja wartości AUC (`direction='maximize'`). Następnie wykonuje dwieście pięćdziesiąt iteracji, gdzie z każdą iteracją na nowo jest losowana wartość hiperparametru C i liczona wartość AUC. Po dwustu pięćdziesięciu próbach optymalizacji, w `best_params_optuna` zapisuje wartość hiperparametru C, dla którego model osiąga najwyższą wartość AUC.

Później tworze model regresji logistycznej z wartością C uzyskaną podczas optymalizacji, potem go trenuje i zapisuje przewidywania modelu dla danych testowych oraz gromadzę prawdopodobieństwa przypisania klasy pozytywnej przez model.

```
# =====
#          optymalizacja za pomocą Optuna
# =====
def optuna_optimization(var):
    C = var.suggest_float('C', 1e-4, 1e4, log=True)

    model = LogisticRegression(C=C, penalty='l2', solver='lbfgs', random_state=42, max_iter=1000)
    model.fit(X_train, y_train)
    return roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])

study = optuna.create_study(direction='maximize')
study.optimize(optuna_optimization, n_trials=250)
best_params_optuna = study.best_params

model_optuna = LogisticRegression(**{**best_params_optuna, 'penalty': 'l2', 'solver': 'lbfgs', 'random_state': 42, 'max_iter': 1000})
model_optuna.fit(X_train, y_train)
y_pred_optuna = model_optuna.predict(X_test)
probs_optuna = model_optuna.predict_proba(X_test)[:, 1]
```

Rysunek 25 - fragment kodu implementacji regresji logistycznej - model optymalizowany *optuna*

Rysunek [26] zawiera fragment kodu dotyczący budowania modelu regresji logistycznej optymalizowanej za pomocą random search. Najpierw definiuje zakres przeszukiwać dla hiperparametru C. Następnie inicjalizuje obiekt `RandomizedSearchCV`. `RandomizedSearchCV` jest to jedna z wielu implementacji koncepcji Random Search dostępna w `sklearn`. Regularyzacja L2, algorytm lbfgs, liczba iteracji algorytmu optymalizującego, `random_state`, zakres przeszukiwań hiperparametru C, liczba losowych zestawów hiperparametrów do przetestowania, metryka AUC jako wskaźnik do oceny skuteczności modelu, liczba podziałów walidacji krzyżowej (ang. *cross-validation*), gdzie dane testowe dzielone są na części i jedna z nich służy jedynie do testowania a pozostałe do uczenia oraz ustawienie (`n_jobs`) pozwalające na optymalizację czasową przeszukiwania zakresów parametrów, to parametry służące do przeprowadzenia optymalizacji.

Następnie poszukuje optymalnej wartości hiperparametru C i zapisuje jej wartość w `best_params_random`. Następne kroki są podobne jak w przypadku optymalizacji modelu *optuna*, czyli zbudowanie modelu regresji logistycznej z najlepszą wartością hiperparametru C, trenowanie modelu na zbiorze treningowym, przewidywanie klas dla danych testowych oraz gromadzenie prawdopodobieństw przypisania klasy pozytywnej przez model.


```

# =====
#      optymalizacja za pomocą random search
# =====
randomsearch_optimalization = {
    'C': np.logspace(-3, 3, 20)
}

random_search = RandomizedSearchCV(
    LogisticRegression(penalty='l2', solver='lbfgs', random_state=42, max_iter=1000),
    param_distributions=randomsearch_optimalization,
    n_iter=250,
    scoring='roc_auc',
    random_state=42,
    cv=3,
    n_jobs=-1
)
random_search.fit(X_train, y_train)
best_params_random = random_search.best_params_

model_randomsearch = LogisticRegression(**{**best_params_random, 'penalty':'l2', 'solver':'lbfgs', 'random_state':42, 'max_iter':1000})
model_randomsearch.fit(X_train, y_train)
y_pred_randomsearch = model_randomsearch.predict(X_test)
probs_randomsearch = model_randomsearch.predict_proba(X_test)[: , 1]

```

Rysunek 26 - fragment kodu implementacji regresji logistycznej - model optymalizowany random search

Rysunek [27] przedstawia kod funkcji `evaluate_model()` służącej do oceny skuteczności predykcji modeli za pomocą czterech metryk, tj. dokładności, czułości, precyzji i AUC. Funkcja przyjmuje argumenty: `y_true` - rzeczywiste etykiety klas, `y_pred` - przewidywane etykiety klas przez model, `y_proba` – prawdopodobieństwa przypisania klasy pozytywnej przez model oraz `model_name` czyli nazwa modelu. Dalej obliczane i wyświetlane są te cztery metryki oraz zwracany jest wyniki AUC.

```

# =====
#      ocena modeli - funkcja
# =====
def evaluate_model(y_true, y_pred, y_proba, model_name):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    auc_score = roc_auc_score(y_true, y_proba)
    print(f"\n{model_name} Metrics:")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"AUC: {auc_score:.4f}")
    return auc_score

```

Rysunek 27- fragment kodu implementacji regresji logistycznej - funkcja do oceny modeli

Rysunek [30] zawiera kod funkcji, która służy do wizualizacji macierzy pomyłek. Funkcja przyjmuje argumenty: `y_true`, `y_pred` (opisałem te argumenty powyżej) oraz `title`, czyli tytuł wykresu. Na osiach pojawiają się wartości z `names`, czyli `hepatitis` (zapalenie wątroby) oraz `non hepatitis` (brak zapalenia wątroby). Dalej generuje macierz z biblioteki `sklearn` (`confusion_matrix()`), tworze obiekt wykresu, gdzie `f` – obiekt figury, a `ax` – osie, korzystając z biblioteki `seaborn` rysuje macierz pomyłek jako mapę cieplną (ang. *heatmap*). Na końcu wyświetlam macierz.

Rysunek [29] to fragment kodu, który odpowiada za wywołanie funkcji do oceny modeli oraz wizualizacji macierzy pomyłek (funkcje `evaluate_model()` oraz `Confusion()`).

Rysunek [28] to fragment kodu, który odpowiada za obliczanie wartości potrzebnych do wykresu ROC, czyli FPR oraz TPR. Funkcja `roc_curve()` jest z biblioteki `sklearn` i przyjmuje jako argumenty `y_test` oraz prawdopodobieństwa przypisania klasy pozytywnej przez model. Następnie przy pomocy już dobrze znanej biblioteki `matplotlib` tworze zbiorczy wykres dla trzech modeli, odróżniając je od siebie kolorami. Dodatkowo tworze diagonalną, która

symbolizuje wartość AUC na poziomie 0.5, świadcząca o kiepskiej skuteczności predykcji modelu.

```
# =====  
#                               macierz pomyłek - funkcja  
# =====  
def confusion(y_true, y_pred, title):  
    names = ['Non Hepatitis', 'Hepatitis']  
    cm = confusion_matrix(y_true, y_pred)  
    f, ax = plt.subplots(figsize=(10,10))  
    sns.heatmap(cm, annot=True, linewidth=0.5, linecolor="r", fmt=".0f", ax=ax, cmap='Blues')  
    plt.title(title, size=25)  
    plt.xlabel("Predicted")  
    plt.ylabel("True")  
    ax.set_xticklabels(names)  
    ax.set_yticklabels(names)  
    plt.show()
```

Rysunek 30 - fragment kodu funkcji do wizualizacji macierzy pomyłek

```
# =====  
#                               ocena modeli  
# =====  
auc_no_optimization = evaluate_model(y_test, y_pred_no_optimization, probs_no_optimization, "Logistic Regression (No Optimization)")  
confusion(y_test, y_pred_no_optimization, "Logistic Regression (No Optimization)")  
  
auc_optuna = evaluate_model(y_test, y_pred_optuna, probs_optuna, "Logistic Regression (Optuna)")  
confusion(y_test, y_pred_optuna, "Logistic Regression (Optuna)")  
  
auc_randomsearch = evaluate_model(y_test, y_pred_randomsearch, probs_randomsearch, "Logistic Regression (RandomSearch)")  
confusion(y_test, y_pred_randomsearch, "Logistic Regression (RandomSearch)")
```

Rysunek 29 - fragment kodu implementacji regresji logistycznej - ocena modeli

```
# =====  
#                               wykres ROC  
# =====  
fpr_no_optimization, tpr_no_optimization, _ = roc_curve(y_test, probs_no_optimization)  
fpr_optuna, tpr_optuna, _ = roc_curve(y_test, probs_optuna)  
fpr_randomsearch, tpr_randomsearch, _ = roc_curve(y_test, probs_randomsearch)  
  
plt.figure(figsize=(8, 6))  
plt.plot(fpr_no_optimization, tpr_no_optimization, marker='.', label=f'No Optimization (AUC = {auc_no_optimization:.3f})')  
plt.plot(fpr_optuna, tpr_optuna, marker='.', label=f'Optuna (AUC = {auc_optuna:.3f})')  
plt.plot(fpr_randomsearch, tpr_randomsearch, marker='.', label=f'RandomSearch (AUC = {auc_randomsearch:.3f})')  
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')  
plt.title('ROC Curve Comparison')  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.legend(loc='lower right')  
plt.grid()  
plt.show()
```

Rysunek 28 - fragment kodu implementacji regresji logistycznej - wykres ROC

6. Ewaluacja modeli

Ważnym etapem po implementacji i procesie uczenia modeli jest ocena ich wyników. Żeby wybrać najlepsze modele uczenia maszynowego potrzebuje metryk do oceny ich rezultatów. W mojej pracy wybrałem kilka różnych metryk (podstawowych i bardziej zaawansowanych), które pomogą mi wyselekcjonować kilka najlepszych modeli. W tym rozdziale pochyłę się nad takimi metrykami jak dokładność, czułość czy precyzja, dla najlepszych modeli stworzę macierz pomyłek, która wizualizuje wyniki, krzywe ROC i wartość SHAP, aby pojąć wpływ poszczególnych cech na decyzje modelu [27-29].

❑ Dokładność (ang. *Accuracy*)

Dokładność to najpopularniejszy wskaźnik wykorzystywany do oceny modeli. Ta metryka mówi o tym jaka część próbek ze wszystkich zaklasyfikowanych została zaklasyfikowana poprawnie. Dokładność przyjmuje zakres od 0.00 do 1.00, im wyższa wartość tym lepiej. Dokładność na poziomie 1.00 oznacza, że wszystkie próbki zostały zaklasyfikowane poprawnie, dopasowanie idealne.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Gdzie:

- TP – Prawdziwie pozytywne - liczba przypadków, które zostały poprawnie zakwalifikowane jako pozytywne,
- TN – Prawdziwie negatywne - liczba przypadków, które zostały poprawnie zakwalifikowane jako negatywne,
- FP – Fałszywie pozytywne - liczba przypadków, które zostały błędnie zakwalifikowane jako pozytywne
- FN – Fałszywie negatywne - liczba przypadków, które zostały błędnie zakwalifikowane jako negatywne

❑ Precyzja (ang. *Precision*)

Precyzja dostarcza informacji o tym jak bardzo model jest w stanie przewidywać rzeczywiste klasy dodatnie. Innymi słowy, ile próbek przewidzianych poprawnie faktycznie jest poprawne. Podobnie jak w metryce dokładności zakres wartości to od 0.00 do 1.00, im wartość bliższa 1.00 tym lepiej.

$$Precision = \frac{TP}{TP + FP}$$

❑ Czułość (ang. *Recall*)

Czułość odpowiada na pytanie jaką część stanowią próbki prawidłowo przewidzianych przypadków pozytywnych w odniesieniu do wszystkich przypadków. Zakres wartości czułości przyjmuje podobnie jak dokładność i precyzja, im bliżej 1.00 tym lepiej. Warto mieć na uwadze, że jeżeli model nie oceni żadnej pozytywnej próbki jako błędnej to czułość wyniesie 1.00.

$$Recall = \frac{TP}{TP + FN}$$

❑ Krzywa ROC i AUC (ang. *Receiver Operating Characteristic & Area Under Curve*)

Krzywa ROC ilustruje graficzną relację między czułością (TPR), a specyficznością (FPR). Punkty na tej krzywej przedstawiają w jaki sposób zmienia się efektywność klasyfikatora w zależności od wybranego progu decyzyjnego. Oś pozioma reprezentuje specyficzność, czyli fałszywie pozytywne wyniki, a oś pionowa reprezentuje czułość, czyli odsetek prawidłowo sklasyfikowanych próbek pozytywnych. Im krzywa bliżej górnego lewego rogu wykresu tym lepiej. Jest to jeden z najważniejszych narzędzi do oceny modeli.

Pole pod krzywą ROC, czyli AUC to miara oceny jakości modelu. AUC to prosty, aczkolwiek skuteczny wskaźnik oceny modeli uczenia maszynowego, dający informacje o zdolności modeli do rozróżniania klas. Im wartość AUC bliżej 1.00 tym lepiej.

$$TPR(Recall) = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

❑ Macierz pomyłek (ang. *Confusion matrix*)

W metryce dotyczącej dokładności na początku rozdziału zdefiniowałem co oznaczają skróty TP, TN, NP oraz NF, dlatego będę posługiwał się skróconą formą zapisu. Macierz pomyłek w przypadku klasyfikacji binarnej to tabela z dwoma wierszami i dwoma kolumnami, powstaje z przecięcia klasy przewidywanej oraz klasy rzeczywistej, stąd cztery przypadki TP, TN, NP oraz NF. Macierz pomyłek pozwala zrozumieć, w których miejscach model popełnia błędy.

	Klasa 1 (Przewidywana)	Klasa 0 (Przewidywana)
Klasa 1 (Rzeczywista)	Prawdziwie pozytywne (ang. <i>True Positive</i>)	Fałszywie negatywne (ang. <i>False Negative</i>)
Klasa 0 (Rzeczywista)	Fałszywie pozytywne (ang. <i>False Positive</i>)	Prawdziwie negatywne (ang. <i>True Negative</i>)

Tabela 2 - przykładowa macierz pomyłek

❑ Wartość SHAP (ang. *SHapley Additive exPlanations*)

Dzięki wartości SHAP jestem w stanie się dowiedzieć w jakim stopniu dana cecha wpływa na decyzje modelu. SHAP korzysta z koncepcji teorii gier, żeby określić wkład danej cechy w końcową predykcję. Dzięki tym wartościom wiem, które cechy najistotniej wpłynęły na klasyfikację pacjenta jako chorego lub zdrowego, co czyni tą technikę wartościową w kontekście zastosowania w mojej pracy.

7. Wyniki i dyskusja

Tabela [3] przedstawia podsumowanie wyników dla wszystkich algorytmów, uwzględniając zastosowane techniki optymalizacji (random search oraz optuna). Kolorem zielonym wskazałem trzy modele, które najlepiej poradziły sobie z predykcją.

No.	Algorytm	Technika Optymalizacji	Dokładność	Precyzja	Czułość	AUC
1	Random Forest	Bez optymalizacji	0.6859	0.7552	0.8667	0.4056
2	Random Forest	Optuna	0.6426	0.7656	0.7619	0.5018
3	Random Forest	RandomizedSearchCV	0.6787	0.7553	0.8524	0.4471
4	XGBoost	Bez optymalizacji	0.6282	0.7336	0.8000	0.4329
5	XGBoost	Optuna	0.6643	0.7671	0.8000	0.4833
6	XGBoost	RandomizedSearchCV	0.6787	0.7531	0.8571	0.4259
7	LightGBM	Bez optymalizacji	0.6498	0.7446	0.8190	0.4102
8	LightGBM	Optuna	0.5993	0.7415	0.7238	0.4765
9	LightGBM	RandomizedSearchCV	0.6787	0.7510	0.8619	0.4284
10	SVM	Bez optymalizacji	0.5352	0.7052	0.7189	0.4352
11	SVM	Optuna	0.5668	0.7463	0.7304	0.4404
12	SVM	RandomizedSearchCV	0.5358	0.6950	0.7192	0.4398
13	Logistic Regression	Bez optymalizacji	0.6029	0.7688	0.6810	0.5010
14	Logistic Regression	Optuna	0.6029	0.7688	0.6810	0.5010
15	Logistic Regression	RandomizedSearchCV	0.5812	0.7582	0.6571	0.4986
16	KNN	Bez optymalizacji	0.4188	0.7168	0.3857	0.4732
17	KNN	Optuna	0.4729	0.7581	0.4476	0.5064
18	KNN	RandomizedSearchCV	0.4693	0.7480	0.4524	0.4538
19	GaussianNB	Bez optymalizacji	0.6029	0.7525	0.7095	0.4874
20	GaussianNB	Optuna	0.5993	0.7513	0.7048	0.4878
21	GaussianNB	RandomizedSearchCV	0.6029	0.7525	0.7095	0.4874
22	Decision Tree	Bez optymalizacji	0.5090	0.7606	0.5143	0.5197
23	Decision Tree	Optuna	0.6137	0.7910	0.6667	0.5590
24	Decision Tree	RandomizedSearchCV	0.5704	0.7630	0.6286	0.4954
25	MLP	Bez optymalizacji	0.6173	0.7407	0.7619	0.4889
26	MLP	Optuna	0.6462	0.7772	0.7476	0.5729
27	MLP	RandomizedSearchCV	0.6137	0.7537	0.7286	0.5094
28	NuSVM	Bez optymalizacji	0.6643	0.7489	0.8381	0.4207
29	NuSVM	Optuna	0.5162	0.8333	0.4524	0.6134
30	NuSVM	RandomizedSearchCV	0.7256	0.7519	0.9524	0.4395

Tabela 3 - wykorzystane algorytmy, techniki optymalizacji oraz metryki oceny modeli

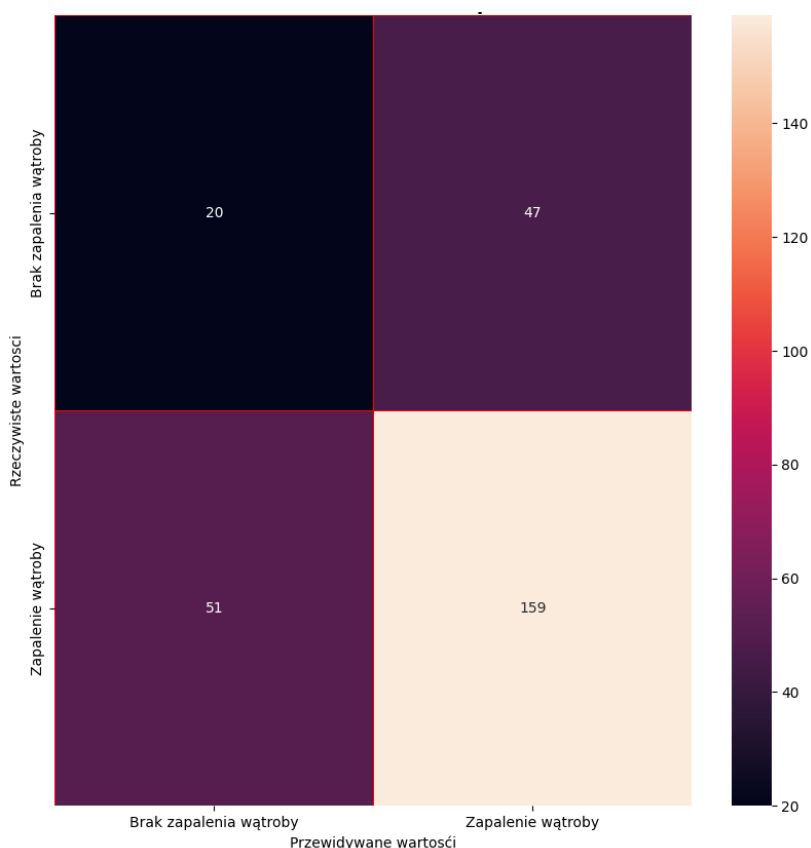
7.1 Wyniki

❑ Las losowy (ang. *Random Forest*)

- Bez optymalizacji: Algorytm Random Forest osiągnął dokładność 0.6859, czułość 0.8667 oraz precyzję 0.7552. Wyniki te wskazują na wysoką skuteczność w klasyfikacji klasy większościowej (zdrowych), lecz niższą efektywność w wykrywaniu klasy mniejszościowej (chorych).
- Optuna: Zastosowanie optymalizacji za pomocą Optuny przyniosło poprawę w balansowaniu wyników między klasami. Dokładność na poziomie 0.6426, czułość wyniosła 0.7619, co świadczy o lepszej zdolności wykrywania klasy mniejszościowej, a precyzja na poziomie 0.7656.
- Random search: Ta technika optymalizacji przyniosła umiarkowane rezultaty, z dokładnością 0.6787, precyzją 0.7553 i czułością 0.8524. Wartości te wskazują, że random search zachował skuteczność w klasyfikacji klasy większościowej, ale nie przyniósł znaczących usprawnień w wykrywaniu klasy mniejszościowej.

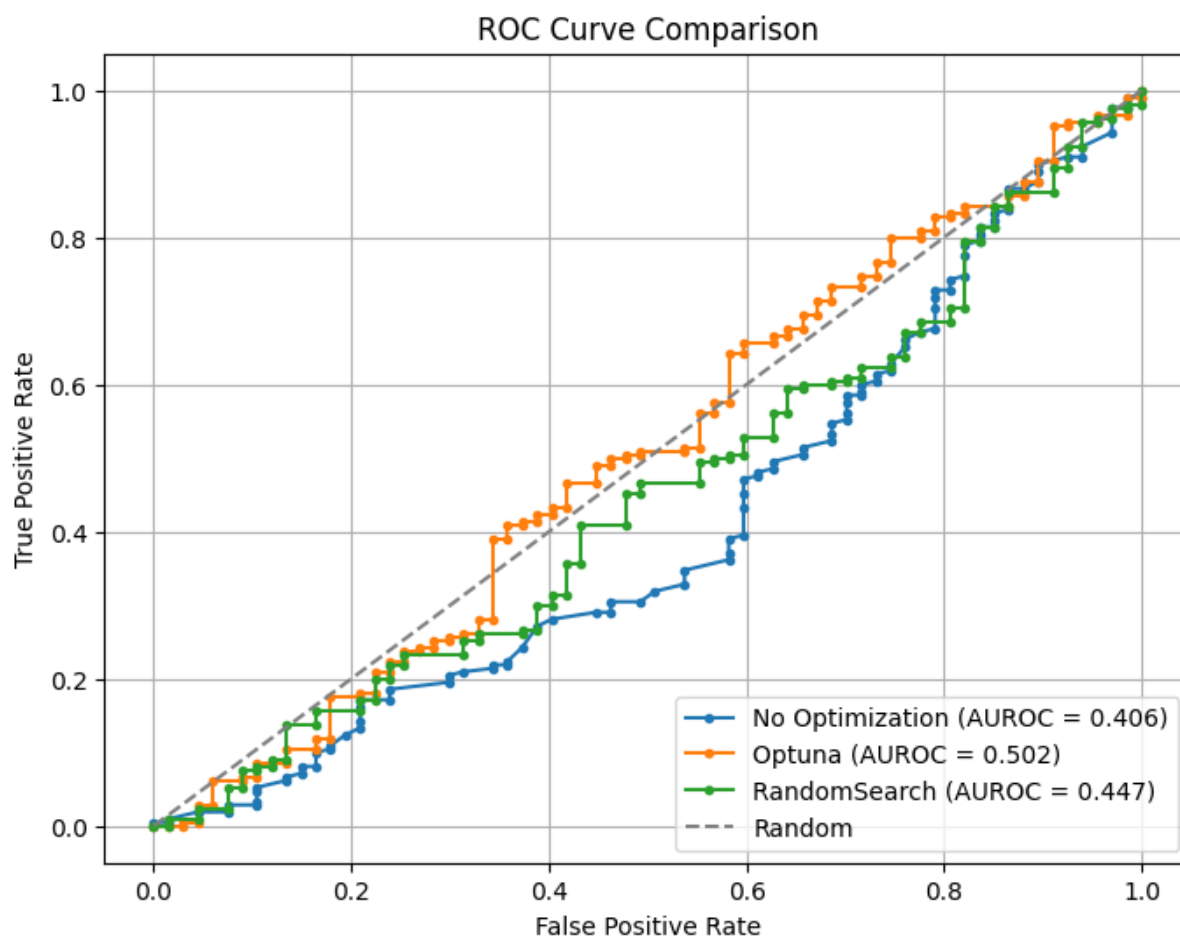
Random Forest okazał się skuteczny w klasyfikacji klasy większościowej, jednak tylko Optuna zapewniła znaczącą poprawę w rozpoznawaniu klasy mniejszościowej.

Rysunek [31] przedstawia macierz pomyłek dla algorytmu lasu losowego, zoptymalizowanego za pomocą Optuny. Na podstawie danych z macierzy wynika, że 47 próbek zaklasyfikowano jako fałszywie negatywne, a 51 próbek jako fałszywie pozytywne, mimo to model osiągnął zadowalającą dokładność.



Rysunek 31 - macierz pomyłek dla lasu losowego z optuną

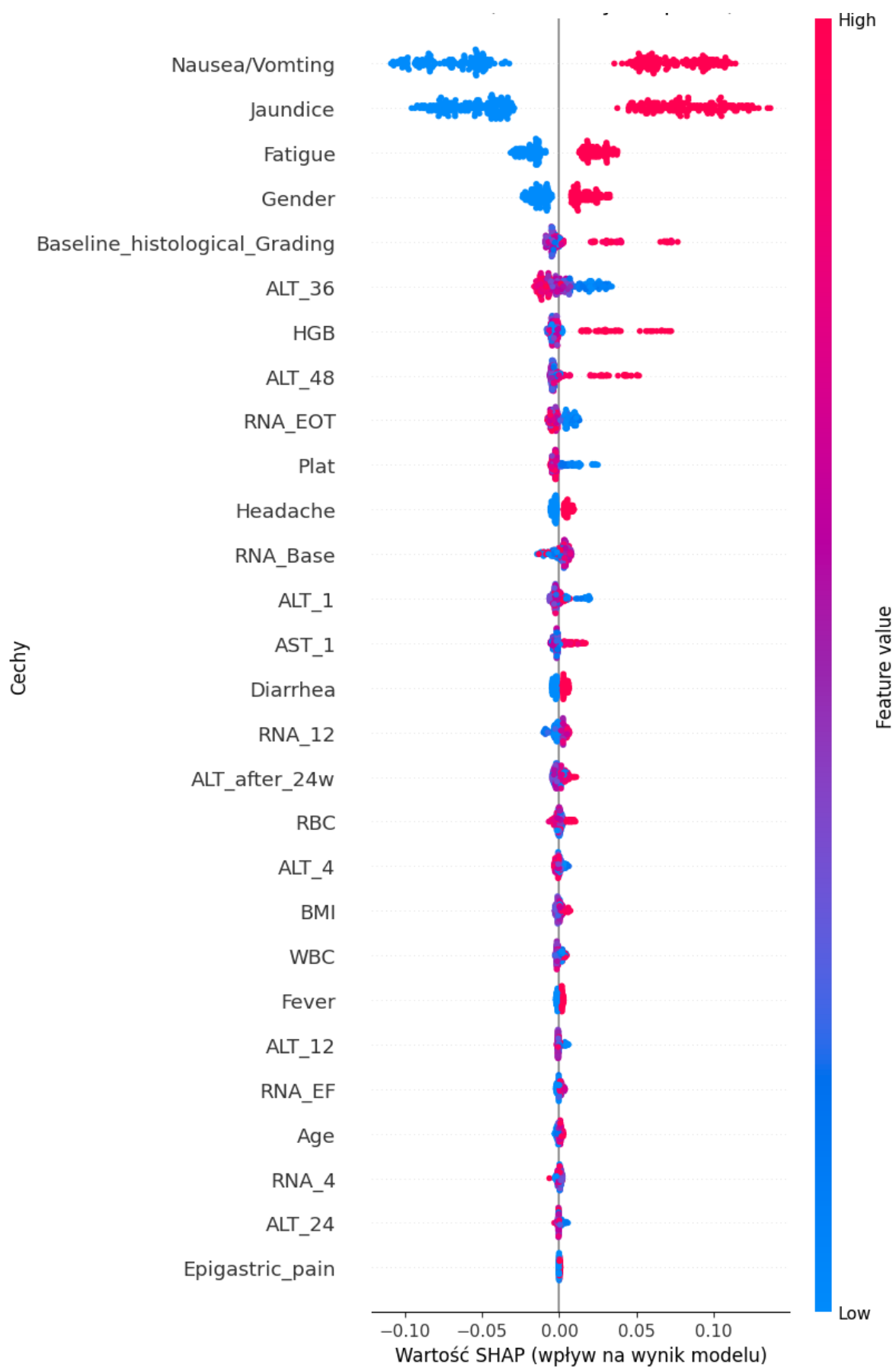
Rysunek [32] przedstawia krzywą ROC dla trzech modeli (bez optymalizacji, dla optuny oraz random search).



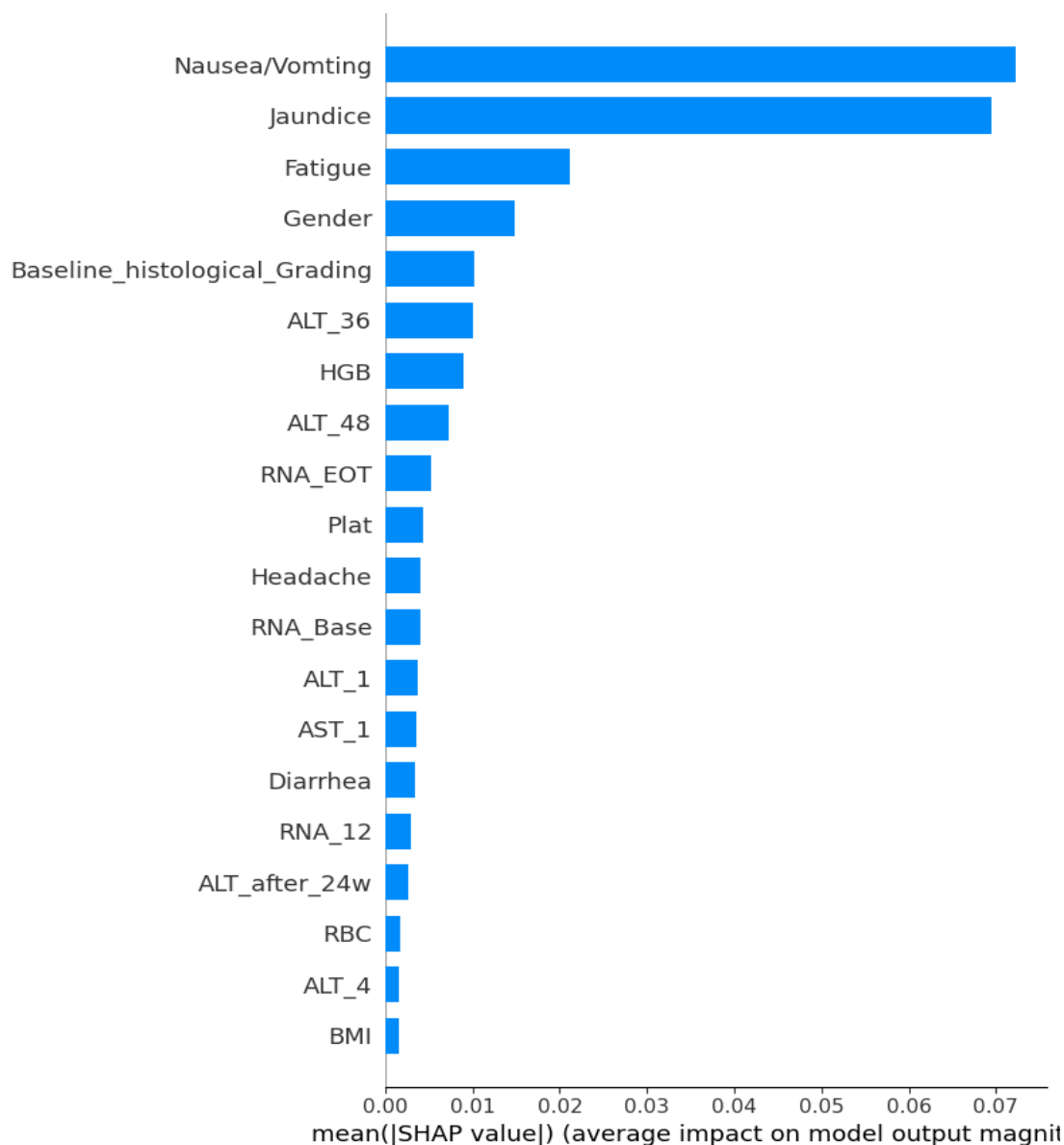
Rysunek 32 - wykres krzywej ROC dla Lasu losowego dla trzech modeli

Rysunek [33] przedstawia wartości SHAP dla modelu lasu losowego zoptymalizowanego za pomocą Optuny. Wykres ukazuje wpływ cech na wyniki modelu. Na osi pionowej znajdują się cechy uznane za najbardziej istotne, a na osi poziomej – wartości SHAP, gdzie wartości dodatnie zwiększają, a ujemne zmniejszają szansę przypisania próbki do określonej klasy. Kolory punktów odzwierciedlają wielkość cechy: czerwony oznacza wysoką wartość, a niebieski – niską, co przekłada się na siłę wpływu na wynik predykcji modelu. Rozrzut punktów pokazuje, jak zróżnicowane wartości cech wpłynęły na predykcję.

Rysunek [34] przedstawia średnie wartości SHAP w formie wykresu słupkowego, wskazując cechy o największym wpływie na predykcję modelu. Na osi pionowej znajdują się cechy posortowane według istotności, a na osi poziomej – średnie wartości SHAP. Wykres ułatwia identyfikację kluczowych cech w analizie predykcyjnej.



Rysunek 33 - wartość SHAP dla lasu losowego z optyką

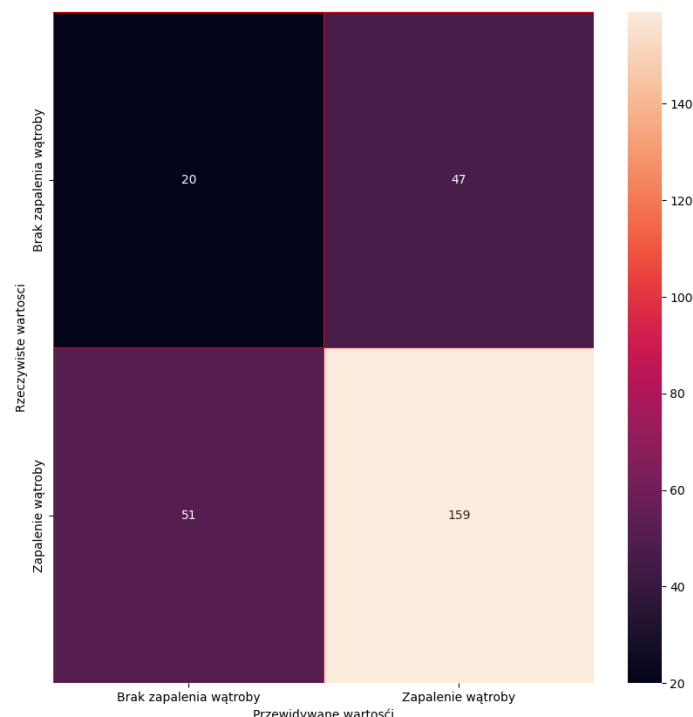


Rysunek 34 - średnia wartość SHAP dla lasu losowego z optuną

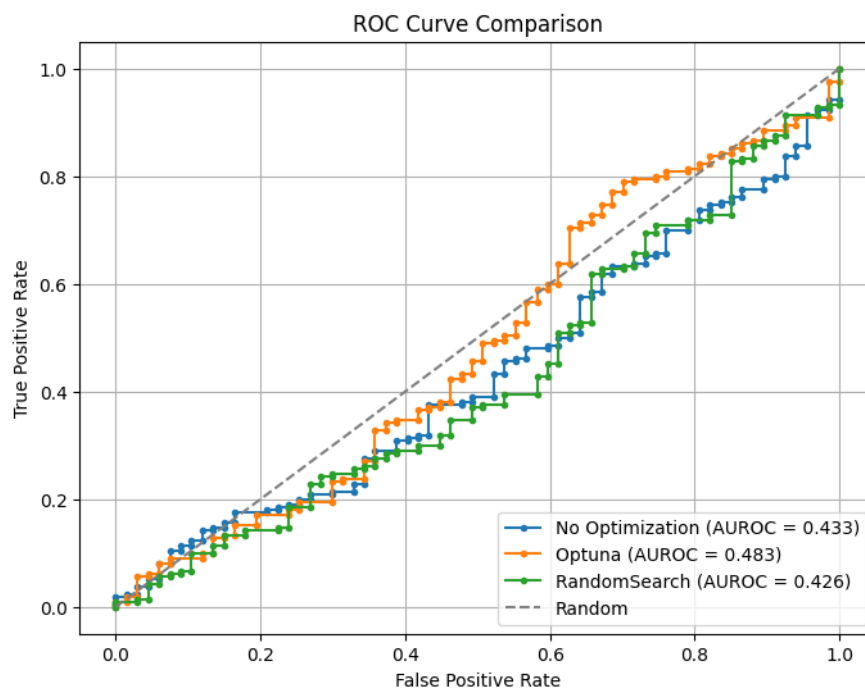
❑ XGBoost

- Bez optymalizacji: Algorytm osiągnął dokładność 0.6282, precyzję 0.7336 oraz czułość 0.8000, co wskazuje na stabilność działania modelu, mimo braku dostrajania hiperparametrów. Wyniki te świadczą o dobrej zdolności modelu do klasyfikacji danych.
- Optuna: Zastosowanie Optuny pozwoliło poprawić wyniki, osiągając dokładność 0.6643, precyzję 0.7671 oraz czułość 0.8000. Wyższa precyzja sugeruje bardziej zrównoważoną klasyfikację, z mniejszą liczbą fałszywie pozytywnych wyników.
- Random search: Dzięki random search uzyskałem dokładność 0.6787, precyzję 0.7531 oraz czułość 0.8571. Wyniki te są porównywalne z wynikami modelu bez optymalizacji, choć zauważalna jest lepsza czułość, co wskazuje na poprawę wykrywania klasy mniejszościowej.

XGBoost okazał się najskuteczniejszy w połączeniu z Optuną, co podkreśla znaczenie tej techniki w optymalizacji złożonych modeli predykcyjnych.



Rysunek 36 - macierz pomyłek dla modelu XGBoost w połączeniu z optuną



Rysunek 35 - wykres krzywej ROC dla modelu XGBoost z różnymi technikami optymalizującymi

Rysunek [36] przedstawia macierz pomyłek dla algorytmu XGBoost w połączeniu z optuną jako techniką optymalizującą. Model zaklasyfikował 51 próbek jako prawdziwie negatywnych oraz 47 próbek jako fałszywie pozytywne, mimo to model osiąga zadawalającą dokładność.

Rysunek [35] przedstawia krzywą ROC dla trzech modeli (bez optymalizacji, dla optuny oraz randomizedsearchcv). Wskaźnik AUC dla modelu z optuną sugeruje umiarkowaną wydajność modelu w rozróżnianiu klas.

❑ LightGBM

- Bez optymalizacji: Model LightGBM osiągnął dokładność 0.6498, precyzję 0.7446 oraz czułość 0.8190. Wyniki te są zbliżone do wyników uzyskanych przez XGBoost, wskazując na stabilność działania modelu bez dostrajania parametrów.
- Optuna: Zastosowanie Optuny pozwoliło uzyskać czułość na poziomie 0.7238, dokładność spadła do 0.5993, a precyzja wyniosła 0.7415. Spadek dokładności może sugerować, że model miał trudności z właściwą klasyfikacją klasy większościowej, co mogło być spowodowane nadmiernym dopasowaniem do klasy mniejszościowej.
- Random search: Optymalizacja za pomocą random search przyniosła dokładność na poziomie 0.6787, precyzję 0.7510 oraz czułość 0.8619. Wyniki te pokazują dobrą równowagę między metrykami.

LightGBM wyróżnił się stabilnością w każdej konfiguracji optymalizacyjnej. Jednakże różnice w wynikach między zastosowanymi technikami optymalizacji były relatywnie niewielkie, co może sugerować, że model jest mniej wrażliwy na dobór hiperparametrów w porównaniu z innymi algorytmami.

❑ SVM

- Bez optymalizacji: Algorytm SVM osiągnął dokładność na poziomie 0.5352, precyzję 0.7052 oraz czułość 0.7189. Wyniki te wskazują na przyzwoitą wydajność modelu w klasyfikacji klasy większościowej, choć skuteczność w rozpoznawaniu klasy mniejszościowej była ograniczona.
- Optuna: Zastosowanie Optuny zwiększyło dokładność do 0.5608, czułość do 0.7304, a precyzję do poziomu 0.7463, co świadczy o poprawie w klasyfikacji klasy mniejszościowej oraz poprawę w identyfikowaniu klasy większościowej.
- Random search: Wyniki tej metody były zbliżone do tych uzyskanych bez optymalizacji – dokładność wyniosła 0.5358, precyzja 0.6950, a czułość 0.7192. Brak istotnych zmian sugeruje, że metoda ta była mniej efektywna w dostrajaniu parametrów SVM.

SVM zoptymalizowany za pomocą Optuny wykazał niewielką poprawę w klasyfikacji klasy mniejszościowej, przy jednoczesnym utrzymaniu stabilnej precyzji. Różnice między zastosowanymi technikami optymalizacji były jednak stosunkowo niewielkie.

❑ Regresja logistyczna

- Bez optymalizacji: Regresja logistyczna uzyskała dokładność 0.6029, precyzję 0.7688 i czułość 0.6810. Te wyniki wskazują na stabilność modelu, który mimo swojej prostoty dobrze radzi sobie z klasyfikacją klasy większościowej.
- Optuna: Optymalizacja za pomocą Optuny utrzymała dokładność, czułość oraz precyzję na niezmiennym poziomie w porównaniu do modelu nie zoptymalizowanego, co zaskakuje z uwagi na zazwyczaj lepsze wyniki modeli optymalizowanych za pomocą optuny.
- Random search: Wyniki tej metody były zbliżone do tych uzyskanych bez optymalizacji – dokładność wyniosła 0.5812, precyzja 0.7582, a czułość 0.6571.

Regresja logistyczna okazała się przyzwoitym modelem o stabilnych wynikach, jednak różnice pomiędzy zastosowanymi technikami optymalizacji były minimalne. Ciekawym

spostrzeżeniem jest to, że wyniki uzyskane za sprawę technik optymalizacyjnych okazały się takie same lub gorsze w porównaniu z nie optymalizowanym algorytmem.

❑ KNN

- Bez optymalizacji: Model KNN uzyskał dokładność na poziomie 0.4188, precyzję 0.7168 oraz czułość 0.3857, co świadczy o bardzo ograniczonej zdolności do rozróżniania klas.
- Optuna: Optymalizacja za pomocą Optuny pozwoliła poprawić czułość do 0.4476, precyzję do poziomu 0.7581 oraz dokładność do 0.4729. Wyniki sugerują nieznaczna poprawę w stosunku do modelu nie optymalizowanego.
- Random search: Technika random search przyniosła umiarkowaną poprawę w wynikach – dokładność wzrosła do 0.4693, precyzja do 0.7480, a czułość osiągnęła poziom 0.4524.

Model KNN zdecydowanie wymagał optymalizacji, aby poprawić jakość klasyfikacji, mimo to jego ogólna wydajność pozostaje bardzo ograniczona.

❑ Naiwny klasyfikator bayesowski

- Bez optymalizacji: Gaussian Naive Bayes osiągnął dokładność 0.6029, precyzję 0.7525 oraz czułość 0.7095, co wskazuje na umiarkowaną skuteczność w klasyfikacji obu klas.
- Optuna: Optymalizacja za pomocą Optuny nie przyniosła poprawy – dokładność wyniosła 0.5993, a precyzja spadła minimalnie do 0.7513, czułość na poziomie 0.7048.
- Random search: Uzyskane wyniki były bardzo zbliżone do tych osiągniętych z Optuną, co sugeruje, że optymalizacja hiperparametrów miała znikomy wpływ na ten model.

Gaussian Naive Bayes okazał się najszybszym modelem pod względem czasu obliczeń, jednak jego jakość predykcji była ograniczona, a optymalizacja hiperparametrów miała minimalny wpływ na wyniki.

❑ Drzewo decyzyjne

- Bez optymalizacji: Drzewo decyzyjne osiągnęło dokładność 0.5090, precyzję 0.7606 oraz czułość 0.5143, co wskazuje na przeciętne wyniki w klasyfikacji klasy mniejszościowej.
- Optuna: Dzięki Optunie wyniki poprawiły się znacząco – dokładność wzrosła do 0.6137, czułość do 0.6667, a precyzja do poziomu 0.7910. Dostrzegam nieznaczna poprawę modelu.
- Random search: Wyniki były stabilne, lecz mniej efektywne niż w przypadku Optuny, co czyni random search mniej skuteczne w tym modelu. Dokładność wyniosła 0.5704, precyzja 0.7630, a czułość 0.6286.

Optymalizacja drzewa decyzyjnego, zwłaszcza za pomocą Optuny, poprawiła jego wydajność. Optymalizacja za pomocą random search poprawiła wyniki modelu bez optymalizacji.

❑ MLP

- Bez optymalizacji: Model MLP osiągnął dokładność 0.6173, precyzję 0.7407 oraz czułość 0.7619, co wskazuje na solidne wyniki jak na model bez optymalizacji.
- Optuna: Optuna poprawiła wyniki – dokładność wzrosła do 0.6462, precyzja do 0.7772, a czułość wyniosła 0.7476. Poprawa ta świadczy o lepszym zrównoważeniu modelu w klasyfikacji obu klas.

- Random search: Model osiągnął dokładność wynoszącą 0.6137, precyzję 0.7537, a czułość 0.7286. Wyniki były nieco lepsze w porównaniu do modelu bez optymalizacji, co sugeruje, że ta metoda nie miała istotnego wpływu na wydajność modelu.

Model MLP był stabilny w każdej konfiguracji, jednak zastosowanie Optuny pozwoliło na zauważalną poprawę wyników.

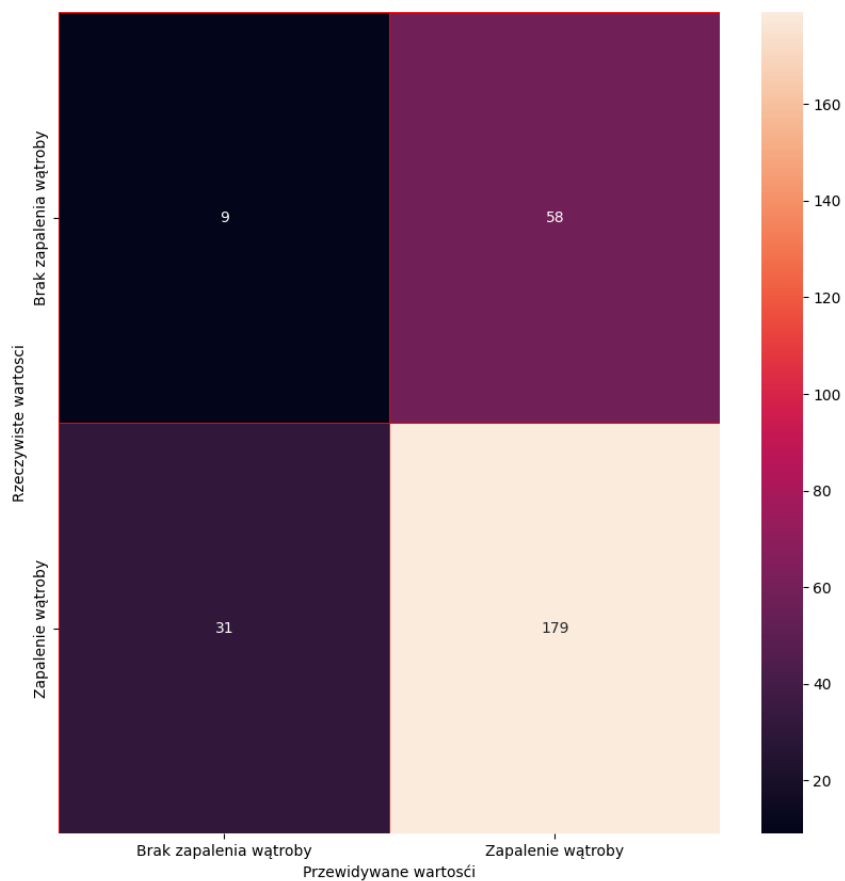
☐ NuSVM

- Bez optymalizacji: NuSVM uzyskał dokładność 0.6643, precyzję 0.7489 oraz czułość 0.8381, co wskazuje na solidne wyniki predykcji modelu.
- Optuna: Wyniki różniły się znacząco – dokładność spadła do 0.5162, jednak precyzja wzrosła do 0.8333, a czułość spadła do 0.4524. Jednak AUC uzyskało wynik aż 0.6134, co czyni ten model jednym z najlepszych.
- Random search: Dokładność wyniosła 0.7256, precyzja 0.7519, a czułość aż 0.9524, co oznacza wyjątkowo dobre wyniki.

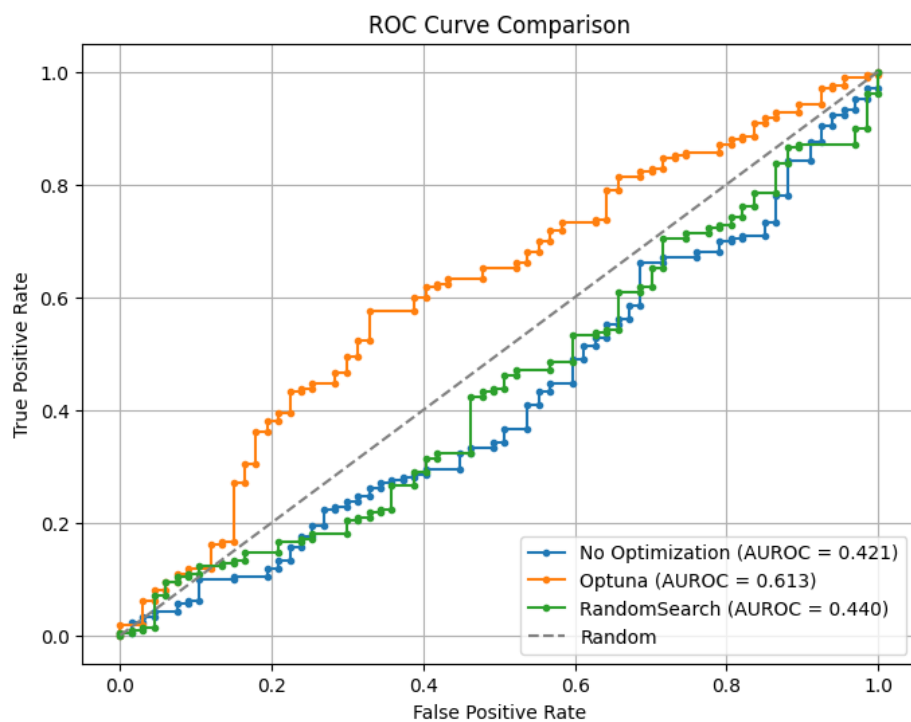
Model NuSVM w połączeniu z random search osiągnął najwyższą czułość, podczas gdy Optuna była bardziej efektywna w zwiększaniu precyzji modelu. Jednakże wynik AUC na poziomie 0.6134 uzyskany przez model NuSVM optymalizowany za pomocą optuny, to jeden z najlepszych modeli tego zestawienia.

Rysunek [37] przedstawia macierz pomyłek dla algorytmu NuSVM zoptymalizowanego za pomocą Optuny. Widać, że model zaklasyfikował 58 próbek jako fałszywie pozytywne oraz 38 próbek jako prawdziwie negatywne. Pomimo tych błędów, model osiągnął zadowalające wyniki, co świadczy o jego przydatności w klasyfikacji.

Rysunek [38] przedstawia krzywą ROC dla trzech wersji modelu NuSVM: bez optymalizacji, z Optuną oraz z random search. Wskaźnik AUC dla modelu zoptymalizowanego za pomocą Optuny wskazuje na umiarkowaną wydajność modelu w rozróżnianiu klas.



Rysunek 37 - macierz pomyłek dla modelu NuSVM w połączeniu z Optuną



Rysunek 38 - wykres krzywej ROC dla trzech modeli algorytmu NuSVM

8. Podsumowanie

Celem mojej pracy było porównanie wyników dziesięciu wybranych algorytmów uczenia maszynowego, które następnie zostały zoptymalizowane za pomocą dwóch technik optymalizujących hiperparametry (optuna i random search), co przełożyło się na zaprojektowanie trzydziestu modeli predykcyjnych oraz wykorzystanie różnych technik związanych ze wstępnym przetwarzaniem danych. Porównanie działania tych modeli zostało przeprowadzone na zbiorze HCV-Egy-Data, który składa się z różnych atrybutów, zarówno z cech klinicznych, jak i deklarowanych przez pacjentów. Uważam, że takie podejście umożliwiło mi przeprowadzenie rzetelnej analizy.

8.1 Przegląd zastosowanych algorytmów

W pracy wykorzystałem kilka zróżnicowanych pod względem działania algorytmów, w tym las losowy, XGBoost, LightGBM, SVM, NuSVM, regresję logistyczną, KNN, GaussianNB, drzewo decyzyjne oraz MLP. Dla każdego z nich przeprowadziłem eksperymenty w trzech wariantach:

- Bez optymalizacji – w celu ustalenia bazowej jakości klasyfikacji.
- Z optymalizacją hiperparametrów przy pomocy random search.
- Z optymalizacją hiperparametrów przy pomocy optuny.

W fazie wstępnego przetwarzania danych skorzystałem z kilku technik, np. z metody SMOTE do balansowania klas oraz poprawy miar takich jak czułość.

Najważniejsze spostrzeżenia:

1. Las losowy, NuSVM oraz XGBoost, te algorytmy w połączeniu z optymalizacją za pomocą optuny okazały się być najlepszymi modelami w moim porównaniu.
2. Zazwyczaj modele które zostały zoptymalizowane osiągały lepsze rezultaty od modeli nieoptymalizowanych, co potwierdza słuszność i wagę stosowania tych technik.
3. NuSVM okazał się konkurencyjny w pewnych konfiguracjach, choć nie uzyskał najwyższej dokładności czy czułości to najlepiej rozróżniał klasy osiągając AUC na poziomie 0.6134.
4. Model NuSVM optymalizowany za pomocą random search uzyskał czułość na poziomie 0.9524, co jest najwyższą wartością z całego porównania.
5. Model NuSVM optymalizowany za pomocą optuny uzyskał precyzję na poziomie 0.8333, co jest najwyższą wartością z całego zestawienia.
6. Model bazowy lasu losowego uzyskał dokładność na poziomie 0.6859, co jest najwyższą osiągniętą wartością przez model w moim porównaniu.
7. Na wyróżnienie zasługują modele drzewa decyzyjnego optymalizowanego za pomocą optuny oraz MLP również optymalizowany optuna które pod względem wyników również nie odstają od trójki najlepszych.
8. Warte uwagi jest fakt, że regresja logistyczna osiągnęła wręcz identyczne wyniki w modelu bazowym oraz optymalizowanym za pomocą optuny.
9. Modele KNN zdecydowanie nie poradziły sobie z zadaniem klasyfikacji binarnej w tym zestawieniu.
10. Zazwyczaj Optuna, mimo dłuższego czasu wykonywania i większego obciążenia sprzętowego okazywała się osiągać lepsze rezultaty niż random search.

8.2 Wnioski dotyczące metod optymalizacji

Zdecydowałem się na wybór i porównanie wyników dwóch technik optymalizujących hiperparametry, czyli random search oraz optuna. Po przeprowadzeniu porównania wyników okazuje się, że był to dobry pomysł, ponieważ zazwyczaj modele, które zostały zoptymalizowane osiągały lepsze rezultaty niż modele niezoptymalizowane. Warty uwagi jest fakt, że optuna zazwyczaj osiągała lepsze rezultaty niż random search, aczkolwiek było to okupione zdecydowanie dłuższym czasem działania i obciążeniem sprzętowym.

Random search z uwagi na swój mechanizm działania, gdzie losowo wybiera wartości parametrów spośród całego zakresu, pozwala to stosunkowo szybko przeszukać cały obszar ustawień. Łatwość implementacji sprawia, że jest to dobra technika optymalizacji, szczególnie przy większych zbiorach.

Natomiast optuna osiągnęła lepsze rezultaty, aczkolwiek w dłuższym czasie. Optuna korzysta z mechanizmów, gdzie bada przestrzeń hiperparametrów tym samym dynamicznie dostosowując strategię przeszukiwania lepszych zakresów parametrów. Dlatego zazwyczaj okazywała się osiągać lepsze rezultaty w porównaniu do random search czy bazowej wersji algorytmu.

Po przeprowadzonej analizie wyników uzyskanych przez modele uważam, że obie wykorzystane przeze mnie techniki optymalizacyjne potwierdziły swoją użyteczność oraz że świadome dostrajanie hiperparametrów ma niebagatelny wpływ jakoś predykcji modeli klasyfikacji w zadaniach medycznych.

8.3 Znaczenie zwiększenia zbioru danych

Wyniki przedstawione w tabeli [3] świadczą o tym, że jakość predykcji modeli jest umiarkowana. Uważam, że warto by:

- Zwiększyć liczbę pacjentów w zbiorze HCV-Egy-Data. Zbiór liczy ponad 1300 pacjentów, co czyni go stosunkowo małym zbiorem jak na standardy w uczeniu maszynowym czy uczeniu głębokim. Pozyskanie większej bazy poprawiłoby jakość predykcji.
- Zdywersyfikować zbiór. Zbiór posiada szerokie spektrum atrybutów, zarówno klinicznych, jak i odczuć pacjentów. Pozyskanie większej liczby zróżnicowanych danych, np. więcej badań laboratoryjnych, zmiennych demograficznych oraz odczuć pacjentów wpłynęłoby pozytywnie na jakość predykcji modelu.

8.4 Kierunki dalszego rozwoju i rekomendacje

Dokonanie analizy uzyskanych wyników oraz zestawienie je w przystępnej formie, w tabeli [3] uświadomiło mi, że jakość predykcji tych modeli nie jest zadowalająca. Skupiłem się na temacie, co warto zmienić, aby osiągnąć lepsze rezultaty. Poniższa lista to moje sugestie nad którymi warto się zastanowić.

1. Większy rozmiar zbioru oraz dywersyfikacja. Te zagadnienia poruszam we wcześniejszym podrozdziale.
2. Wielokrotna walidacja krzyżowa. Warto byłoby zastosować wielokrotną walidację krzyżową, która polega na podzieleniu zbioru danych na równe części (ang. *folds*), gdzie

jedna z części służy do testowania a reszta do uczenia modelu. Części uczące i testowe się zmieniają. Takie podejście pozwala wykorzystać cały zbiór danych do nauki i testowania, gdzie wyniki są uśredniane i nie zależą od losowego podziału danych. W efekcie ryzyko przeuczenia modelu zmniejsza się, zostaje poprawiona stabilność oraz lepsze wykorzystanie danych [30].

3. Więcej technik optymalizacyjnych. W pracy skupiłem się na dwóch popularnych metodach, czyli *optune* i *random search*. Warto byłoby sprawdzić jakie wyniki modele osiągną przy takich metodach optymalizacji hiperparametrów jak przeszukiwanie siatki (ang. *Grid search*), optymalizacji bayesowskiej (ang. *Bayesian optimization*) czy hiperpaśmie (ang. *hyperband*).
4. Normalizacja. Uważam, że warto byłoby porównać wyniki uzyskane za pomocą normalizacji z wynikami osiągniętymi za pomocą standaryzacji. Kiepski wynik modeli KNN mógł być spowodowany tym, że algorytm lepiej radzi sobie z danymi znormalizowanymi [31].
5. Redukcja wymiarowości. Twierdzę, że warto by zastosować techniki takie jak PCA (ang. *Principal Komponent Analysis*) lub ICA (ang. *Independent Komponent Analysis*) w celu zmniejszenia liczby cech i zachowania tych które mają większe znaczenie dla modelu [32].
6. Więcej algorytmów uczenia maszynowego. Istotne może być zbudowanie i porównanie wyników jakie uzyskają algorytmy uczenia maszynowego, które być może lepiej poradzą sobie z klasyfikacją binarną.
7. Więcej metryk oceny modeli. Skupiłem się na czterech metrykach: czułość, dokładność, precyzja oraz AUC. Dla najlepszych modeli stworzyłem macierz pomyłek oraz wykres SHAP. Dodanie innych metryk oceny skuteczności jak wynik *f1* (ang. *F1 score*) czy strata Hamminga (ang. *Hamming loss*), mogłyby dać informacje, dlaczego algorytm nie radzi sobie z klasyfikacją.
8. Dłuższą optymalizacją parametrów. W kontekście technik optymalizacyjnych większy zakres parametrów daje szerszy zakres do poszukiwań optymalnych ustawień. Również zwiększenie liczby iteracji może mieć znaczący wpływ na wyniki predykcji.

Poruszyłem kwestie zarówno związane ze zbiorem danych, technikami wstępnego przetwarzania, metodami związanymi z procesem uczenia, optymalizacji i oceny wyników. Te sugestie zaaplikowane, powinny znacząco wpłynąć na wynik predykcji i dać jeszcze lepsze rezultaty niż osiągnięte przeze mnie.

8.5 Podsumowanie końcowe

Po przeprowadzeniu porównania trzydziestu modeli dochodzę do wniosku, że przewidywanie stadium chorób wątroby, opierając się na nieinwazyjnych danych (biochemicznych oraz demograficznych) może być z powodzeniem realizowane wykorzystując modele uczenia maszynowego. Modele algorytmów lasu losowego, XGBoost czy NuSVM wspierane przez *optune* osiągają zadowalające wyniki poszczególnych metryk (czułość, precyzja, dokładność oraz AUC), jak i dobrą ogólną zdolność predykcyjną. Inne algorytmy takie jak KNN, regresja logistyczna czy LightGBM nie zaskoczyły mnie pozytywnie, osiągając znacznie gorsze wyniki.

Uważam, że otrzymane wyniki spełniają założenie tej pracy, które polegało na porównaniu i ocenie przydatności poszczególnych modeli do predykcji stadiów chorób wątroby. Mimo to, należałoby dalej rozwijać opisane podejście, aby wykorzystać potencjał tych

algorytmów, poprzez wdrożenie 10-krotnej walidacji krzyżowej, PCA lub ICA, poszerzenia zbioru danych, pozyskania większej ilości algorytmów uczenia maszynowego i technik optymalizacji hiperparametrów oraz dodania kolejnych metryk oceny skuteczności modeli. Te kroki przełożą się na znaczną poprawę wyników, co w konsekwencji przełoży się na polepszenie opieki nad pacjentami i wsparciem placówek medycznych, w tym i lekarzy.

Chciałbym podkreślić, że zastąpienie biopsji nieinwazyjnym podejściem staje się coraz bardziej realne. Przeprowadzona przeze mnie analiza może stanowić podstawy do dalszej pracy nad praktycznym systemem, który szybciej i precyzyjniej wykrywałby zaawansowane stadia chorób wątroby.

Bibliografia

1. Sebastian Patrzyk, Anna Woźniacka, Sztuczna inteligencja w medycynie, Łódź: Wydawnictwo Uniwersytetu w Łodzi, 2022
2. Anna Kłeczka, Bogdan Mazur, Krzysztof Tomaszek, Radosław Dzik, Agata Kabała-Dzik, Diagnostyka zwłóknienia wątroby z wykorzystaniem digitalizacji i analizy cyfrowej, Diagnostyka Laboratoryjna Tom: 59, nr: 2, 2023
3. Marek Matacz, SI potrafi diagnozować chorą wątrobę, Nauka w Polsce, 2024
4. Alicja Kapiszka, AI-sztuczna inteligencja w diagnozowaniu chorób, dr zdrowie, url: <https://drzdrowie.pl/ai/> (Dostęp: 16.01.2025)
5. Zbiór danych wykorzystany w pracy, url: <https://archive.ics.uci.edu/dataset/503/hepatitis+c+virus+hcv+for+egyptian+patients> (Dostęp: 16.01.2025)
6. Sztuczna inteligencja w medycynie, url: <https://www.univio.com/pl/blog/sztuczna-inteligencja-w-medycynie-przyklady/> (Dostęp: 16.01.2025)
7. The evolution of ai in healthcare, xsolis, url: https://www-xsolis-com.translate.goog/blog/the-evolution-of-ai-in-healthcare/?_x_tr_sl=en&_x_tr_tl=pl&_x_tr_hl=pl&_x_tr_pto=rq&_x_tr_hist=true (Dostęp: 16.01.2025)
8. Jarosław Gośliński, Teleporady i telemedycyna w dobie pandemii covid-19, url: <https://www.zwrotnikraka.pl/teleporady-i-telemedycyna/> (Dostęp: 16.01.2025)
9. Sztuczna inteligencja pomoże opracować szczepionki na nowe mutacje koronawirusa, wywiad z prof. Jackiem Błazewiczem, url: <https://innowacje.newseria.pl/news/sztuczna-inteligencja,p1487885086> (Dostęp: 16.01.2025)
10. Ewa Zielińska, Sztuczna inteligencja w medycynie, 2024, url: <https://www.wsb.net.pl/technologie/sztuczna-inteligencja-w-medycynie/> (Dostęp: 16.01.2025)
11. Józef Kielar, Gawrońska: Do stosowania sztucznej inteligencji w medycynie niezbędne są precyzyjne przepisy, 2024, url: <https://www.prawo.pl/zdrowie/sztuczna-inteligencja-w-medycynie-wyzwania-dla-polski,529156.html> (Dostęp: 16.01.2025)
12. Ewa Krok, Pozyskiwanie i gromadzenie danych medycznych - analiza jakości, Uniwersytet Szczeciński, url: <https://bazekon.uek.krakow.pl/en/gospodarka/171520121> (Dostęp: 16.01.2025)
13. Anna Wieczorek, Sztuczna inteligencja w służbie medycynie, url: <https://botland.com.pl/blog/sztuczna-inteligencja-w-sluzbie-medycynie/#Przewidywanie-diagnoza-kontrola> (Dostęp: 16.01.2025)
14. Comarch Healthcare, Machine learning w medycynie - zalety rozwiązania i przykłady zastosowania według doświadczeń Comarch Healthcare, url: <https://www.rynekzdrowia.pl/E-zdrowie/Machine-Learning-w-medycynie-zalety-rozwiazania-i-przyklady-zastosowania-wedlug-doswiadczen-Comarch-Healthcare,187062,7.html> (Dostęp: 16.01.2025)
15. Konrad Wojdan, Marcin Moniuszko, Sztuczna Inteligencja w medycynie - stan aktualny i wyzwania wdrożeniowe, 2022
16. KGoldenOwls, Deep learning I jego możliwości, url: <https://goldenowls.pl/deep-learning-i-jego-mozliwosci/> (Dostęp: 16.01.2025)
17. Narodowy Instytut Zdrowia Publicznego, Wirusowe zapalenie wątroby typu C - informacje ogólne, url: <https://epibaza.pzh.gov.pl/story/wirusowe-zapalenie-watroby-typu-c-informacje-ogolne> (Dostęp: 16.01.2025)

18. Ministerstwo Cyfryzacji, Standard techniczny, 2018, url:
https://dane.gov.pl/media/ckeditor/2018/12/05/standard-techniczny_22-11-2018.pdf
(Dostęp: 16.01.2025)
19. Jurand Bień, Uczenie maszynowe: obserwacje odstające, 2022, url:
<https://ekordo.pl/uczenie-maszynowe-observacje-odstajace/> (Dostęp: 16.01.2025)
20. Likebupt, PeterCLu, v-chmcl, SMOTE, 2024, url: <https://learn.microsoft.com/pl-pl/azure/machine-learning/component-reference/smote?view=azureml-api-2> (Dostęp: 16.01.2025)
21. Marcin Sydow, Eksploracja danych wstępne przetwarzanie danych, url:
<https://users.pja.edu.pl/~msyd/ewd/preprocessing.pdf> (Dostęp: 16.01.2025)
22. Aleksandra Król-Nowak, Katarzyna Kotarba, Podstawy uczenia maszynowego, Wydawnictwo: AGH, Kraków 2022, url:
https://winntbg.bg.agh.edu.pl/skrypty4/0599/podstawy_uczenia.pdf (Dostęp: 16.01.2025)
23. Rodzaje ML i najpopularniejsze algorytmy, url: <https://www.datasciencierobie.pl/rodzaje-ml-i-najpopularniejsze-algorytmy/> (Dostęp: 16.01.2025)
24. Nikhil Kumar, Boost your machine learning: exploring XGBoost vs LightGBM, 2024, url: <https://www.linkedin.com/pulse/boost-your-machine-learning-exploring-xgboost-vs-lightgbm-kumar-2djdc> (Dostęp: 16.01.2025)
25. Aniruddhan26, Underrated and implementing Nu-Support Vector Classification (NuSVC), 2024, url: <https://www.geeksforgeeks.org/underrated-and-implementing-nu-support-vector-classification-nusvc/> (Dostęp: 16.01.2025)
26. Abhishek Jain, Optuna vs GridSearchCV vs RandomSearchCV— Hyperparameter Tuning Techniques. 2025, url: <https://medium.com/@abhishekjainindore24/optuna-vs-gridsearchcv-vs-randomsearchcv-hyperparameter-tuning-techniques-ea8e2ada28d0> (Dostęp: 16.01.2025)
27. Krystyna Piątkowska, macierz błędów i co z tego wynika, url:
<https://www.statystyczny.pl/macierz-bledow-raport-dokladnosc-czulosc-precyzja/>
(Dostęp: 16.01.2025)
28. Grzegorz Harańczyk, Krzywe Roc, czyli ocena jakości klasyfikatora i poszukiwanie optymalnego punktu odcięcia, StatSoft Polska sp. z o.o., url:
https://media.statsoft.pl/_old_dnn/downloads/krzywe_roc_czyli_ocena_jakosci.pdf
(Dostęp: 16.01.2025)
29. Qlik.com, Omówienie ważności SHAP w uczeniu eksperymentów, url:
https://help.qlik.com/pl-PL/cloud-services/Subsystems/Hub/Content/Sense_Hub/AutoML/shap-importance.htm (Dostęp: 16.01.2025).
30. Adrian Horzyk, walidacja krzyżowa, url:
<https://home.agh.edu.pl/~horzyk/lectures/miw/MIW-WalidacjaKrzyzowa.pdf> (Dostęp: 16.01.2025)
31. Eitca, Jaki jest cel skalowanie w uczeniu maszynowym I dlaczego jest to ważne?, 2023, url: <https://pl.eitca.org/artificial-intelligence/eitc-ai-mlp-machine-learning-with-python/regression/pickling-and-scaling/examination-review-pickling-and-scaling/what-is-the-purpose-of-scaling-in-machine-learning-and-why-is-it-important/> (Dostęp: 16.01.2025)
32. Adrian Horzyk, Transformacje i jakość danych, url:
<https://home.agh.edu.pl/~horzyk/lectures/miw/MIW-TransformacjeJakoscDanych.pdf>
(Dostęp: 16.01.2025)

Spis rysunków

Rysunek 1 - schemat eksperymentu	8 -
Rysunek 2 - histogram obrazujący grupy wiekowe pacjentów.....	14 -
Rysunek 3 - wykres kołowy przedstawiający rozkład płci w badaniu	15 -
Rysunek 4 - wykres słupkowy obrazujący rozkład płci względem stadium choroby.....	15 -
Rysunek 5 - wykres słupkowy przedstawiający rozkład objawów w badaniu	16 -
Rysunek 6 - histogram przedstawiający rozkład BMI w badaniu.....	16 -
Rysunek 7 - wykres pudełkowy obrazujący rozkład WBC względem płci w badaniu	17 -
Rysunek 8 - wykres pudełkowy obrazujący rozkład WBC w zależności od stadium choroby ..	17 -
Rysunek 9 - wykres obrazujący rozkład poziomu hemoglobiny względem stadiów choroby...	18 -
Rysunek 10 - wykres obrazujący rozkład enzymu ALT w różnych stadiach choroby	18 -
Rysunek 11 - wykres obrazujący skorelowanie cech numerycznych w zbiorze danych	19 -
Rysunek 12 - heatmapa zależności między liczbą czerwonych krwinek a poziomem enzymu AST.....	20 -
Rysunek 13 - heatmapa zależności między liczbą białych krwinek a poziomem enzymu AST ..	20 -
Rysunek 14 - wykres heksagonalny obrazujący skorelowanie WBC z enzymem ALT w pierwszym tygodniu od rozpoczęcia leczenia.....	21 -
Rysunek 15 - wykres heksagonalny obrazujący skorelowanie WBC z enzymem ALT po 48 tygodniach od rozpoczęcia badania.....	21 -
Rysunek 16 - wykres heksagonalny obrazujący skorelowanie WBC z enzymem ALT po 24 tygodniach od zakończenia leczenia	22 -
Rysunek 17 - wykres heksagonalny obrazujący skorelowanie RBC z enzymem ALT w pierwszym tygodniu od rozpoczęcia leczenia.....	22 -
Rysunek 18 - wykres heksagonalny obrazujący skorelowanie RBC z enzymem ALT po 48 tygodniach od rozpoczęcia leczenia	23 -
Rysunek 19 - wykres heksagonalny obrazujący skorelowanie RBC z enzymem ALT po 24 tygodniach od zakończenia leczenia	23 -
Rysunek 20 - wykres heksagonalny obrazujący skorelowanie liczby płytek krwi z enzymem ALT po 48 tygodniach od rozpoczęcia leczenia	24 -
Rysunek 21 - wykres heksagonalny obrazujący skorelowanie liczb płytek krwi z enzymem ALT po pierwszym tygodniu od rozpoczęcia leczenia	24 -
Rysunek 22 - wykres heksagonalny obrazujący skorelowanie liczby płytek krwi z enzymem ALT po 24 tygodniach od zakończenia leczenia	25 -
Rysunek 23 - fragment kodu implementacji regresji logistycznej – importowane biblioteki-	29 -
Rysunek 24 - fragment kodu implementacji regresji logistycznej - model nie optymalizowany-	29 -
Rysunek 25 - fragment kodu implementacji regresji logistycznej - model optymalizowany optuną	30 -
Rysunek 26 - fragment kodu implementacji regresji logistycznej - model optymalizowany random search	31 -
Rysunek 27- fragment kodu implementacji regresji logistycznej - funkcja do oceny modeli-	31 -
Rysunek 30 - fragment kodu implementacji regresji logistycznej - wykres ROC.....	32 -
Rysunek 28 - fragment kodu implementacji regresji logistycznej - ocena modeli	32 -

Rysunek 29 - fragment kodu funkcji do wizualizacji macierzy pomyłek	- 32 -
Rysunek 31 - macierz pomyłek dla lasu losowego z optuną	- 36 -
Rysunek 32 - wykres krzywej ROC dla Lasu losowego dla trzech modeli.....	- 37 -
Rysunek 33 - wartość SHAP dla lasu losowego z optuną	- 38 -
Rysunek 34 - średnia wartość SHAP dla lasu losowego z optuną	- 39 -
Rysunek 35 - wykres krzywej ROC dla modelu XGBoost z różnymi technikami optymalizującymi	- 40 -
Rysunek 36 - macierz pomyłek dla modelu XGBoost w połączeniu z optuną.....	- 40 -
Rysunek 37 - macierz pomyłek dla modelu NuSVM w połączeniu z Optuną	- 44 -
Rysunek 38 - wykres krzywej ROC dla trzech modeli algorytmu NuSVM.....	- 44 -

Spis tabel

Tabela 1 - parametry wykorzystane w niniejszej pracy	- 11 -
Tabela 2 - przykładowa macierz pomyłek	- 34 -

Tabela 3 - wykorzystane algorytmy, techniki optymalizacji oraz metryki oceny modeli... - 35 -